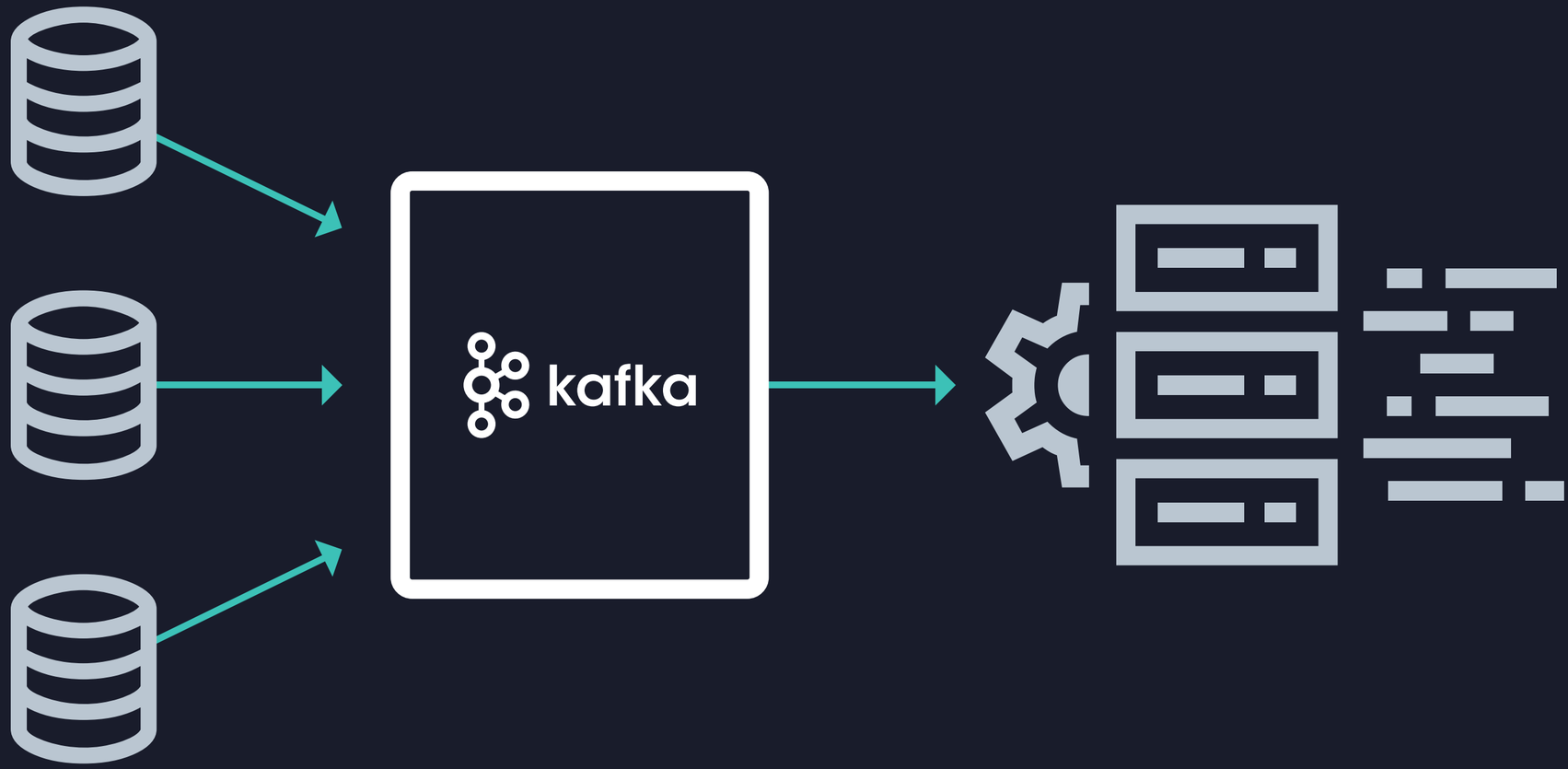
Apache Kafka

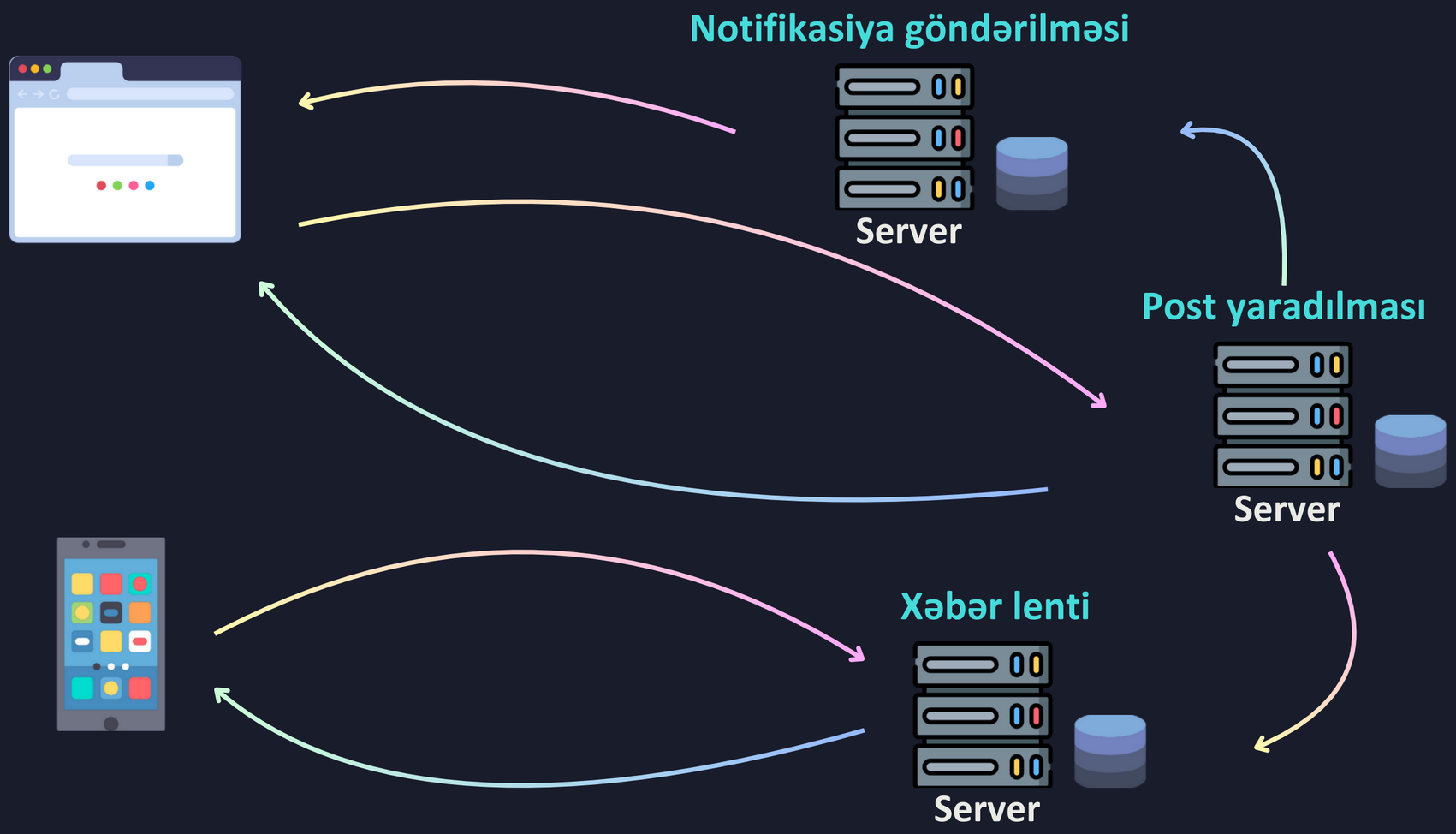
Nəzəriyyədən praktikaya

kafka

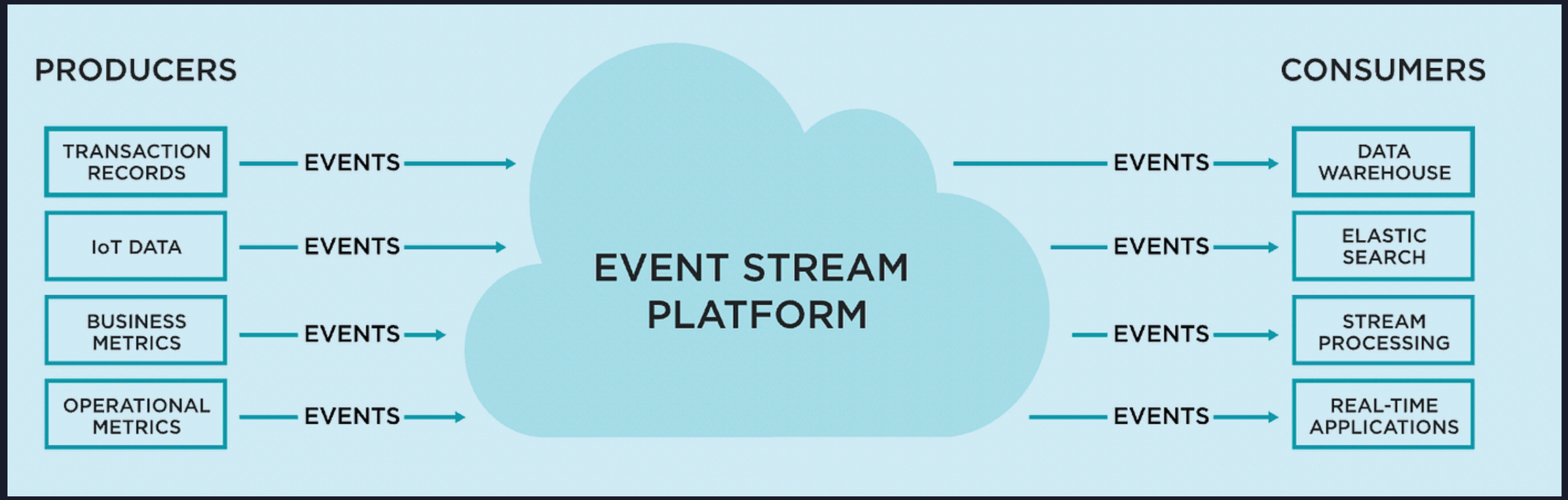
Apache Kafka açıq mənbəli (*open source*), paylanmış (*distrubuted*), hadisə axını (*event streaming*) platformasıdır. Layihənin məqsədi, real vaxt rejimində yüksək ötürmə qabiliyyəti və aşağı gecikmə ilə məlumat axınını emal etmək üçün üfüqi şəkildə miqyaslana bilən (*horizontal scaling*) bir platforma yaratmaqdır.



Paylanmış sistem (*distributed system*) - istifadəçilərə vahid əlaqəli sistem kimi görünən, müstəqil kompüterlər toplusudur. Bu kompüterlər şəbəkə vasitəsilə bir-biri ilə əlaqə saxlayır, resursları və məlumatları paylaşır və tapşırıqları koordinasiya edərək ümumi bir məqsədə çatmağa çalışırlar.



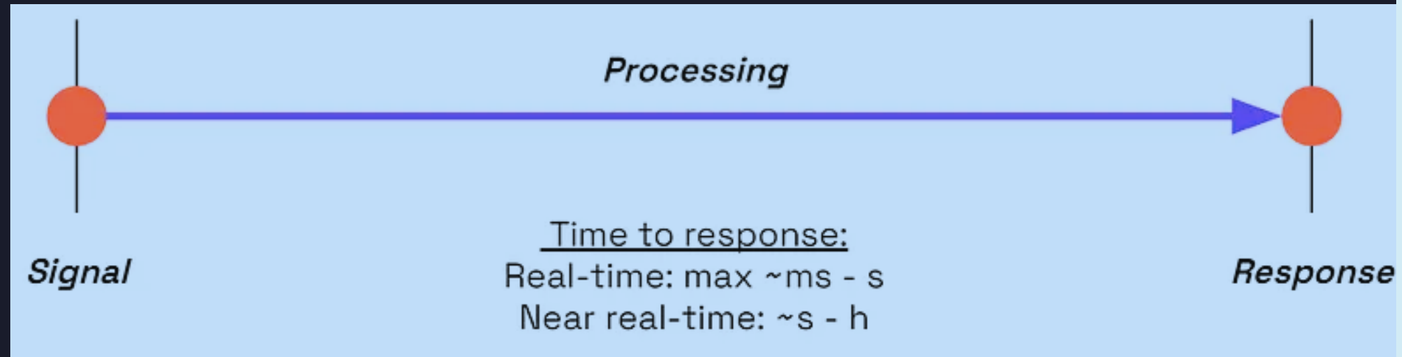
Hadisə (*event*) - proqramda müəyyən bir hadisəni təmsil edir. Bu hadisələr proqram tərəfindən tanınır və adətən asinxron olaraq xarici mühitdən gəlir. Məsələn, istifadəçi klaviaturada bir düyməyə basır və ya mouse ilə klikləyir — bu, bir hadisədir və proqram bu hadisəni tanıyaraq ona reaksiya verə bilər.
Hadisə axını (*event streaming*) - proqramlardan, verilənlər bazalarından və IoT cihazlarından *real vaxt rejimində* məlumatların əldə edilərək saxlanması və ya dərhal emal edilməsi, yaxud təhlil və analitik hesabatlar üçün müxtəlif istiqamətlərə yönləndirilməsidir.

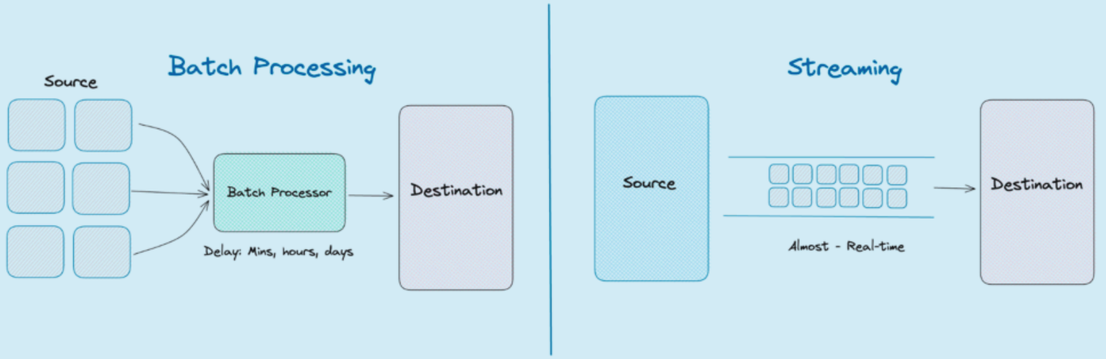


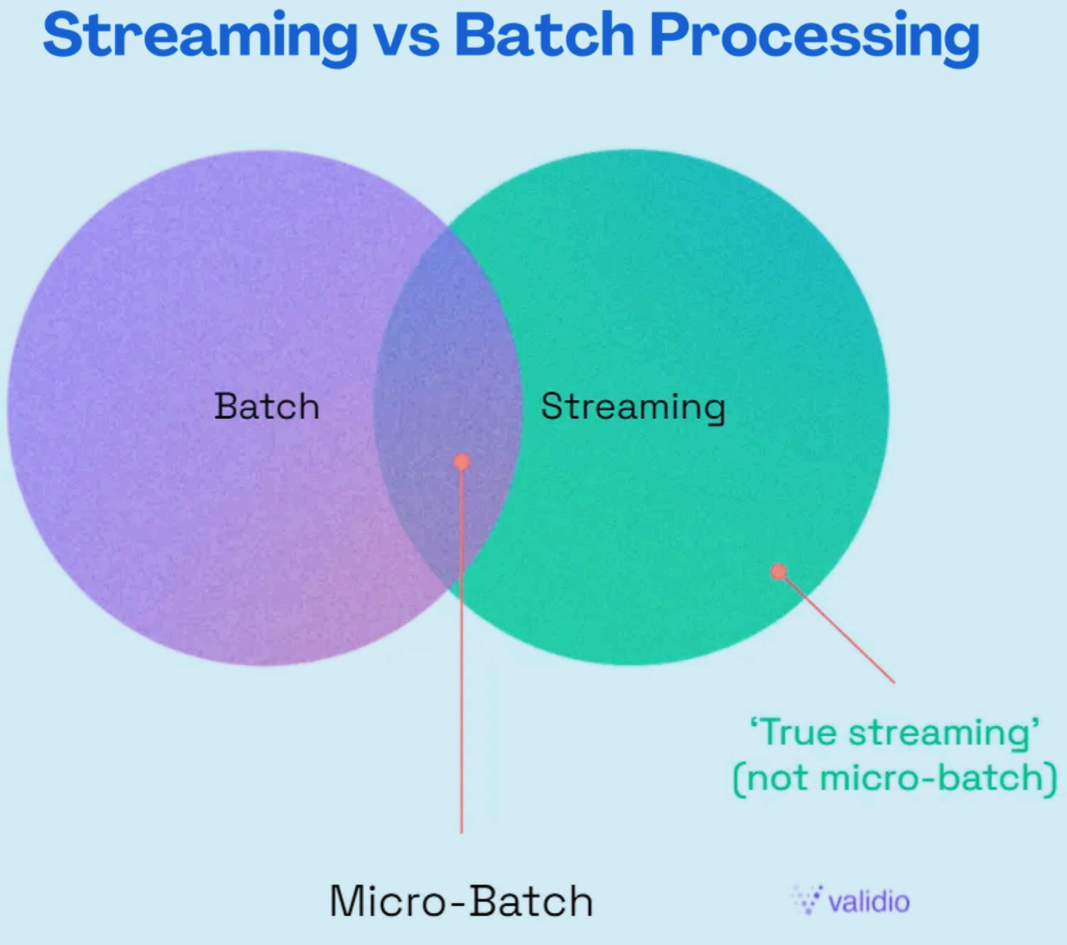
Real vaxt rejimi(*real time*) proqramları adətən “deadline” adlandırılan müəyyən vaxt məhdudiyyətləri daxilində hədisəyə cavab verməyə zəmanət verməlidir. Əgər proqram əvvəlcədən müəyyən olunmuş bu zaman aralığında hadisəyə cavab verməzsə, real-vaxt rejimi uğursuz sayılır. Real vaxt rejimindən danışarkən biz onu iki kateqoriyaya ayıra bilərik:

1. Həqiqi Real vaxt rejimi - bir necə millisaniyədən bir saniyəyə qədər
2. Real vaxta yaxın(*near real time*)- bir neçə saniyədən bir necə saata qədər

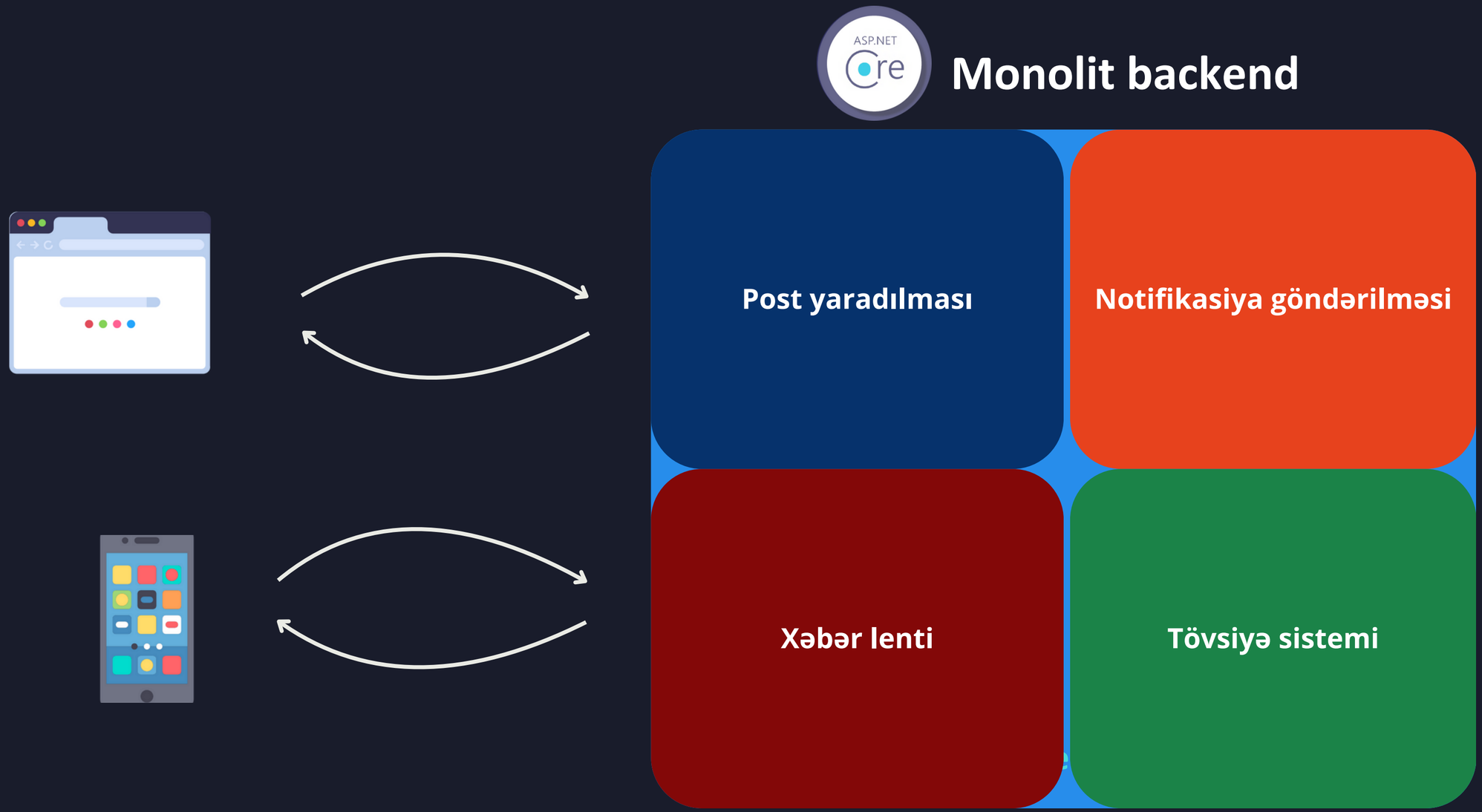
Batch(*qrup,dəstə*) emalı — məlumatların müəyyən zaman intervallarında toplandığı, daha sonra həmin məlumatların toplu şəkildə emal edildiyi, saxlanıldığı və ya başqa bir sistemə göndərildiyi məlumat emalı metodudur. Micro-batch emalı — məlumatların daha kiçik qruplar şəklində toplanıb tez-tez emal edildiyi məlumat emalı metodudur. Ənənəvi batch emalına nisbətən daha tez-tez işləyir, lakin hələ də real-vaxt rejimi ilə tam uyğun gəlmir.



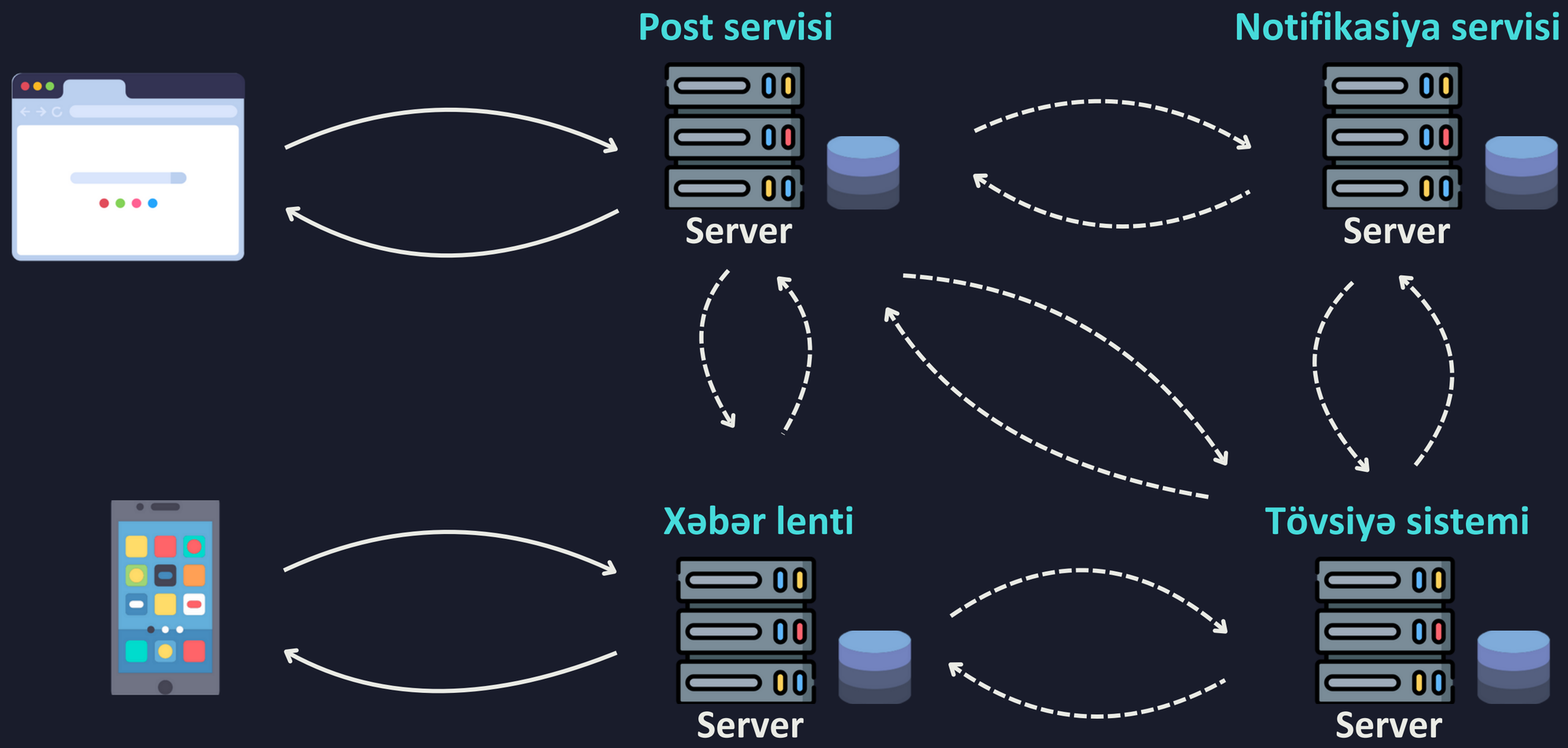




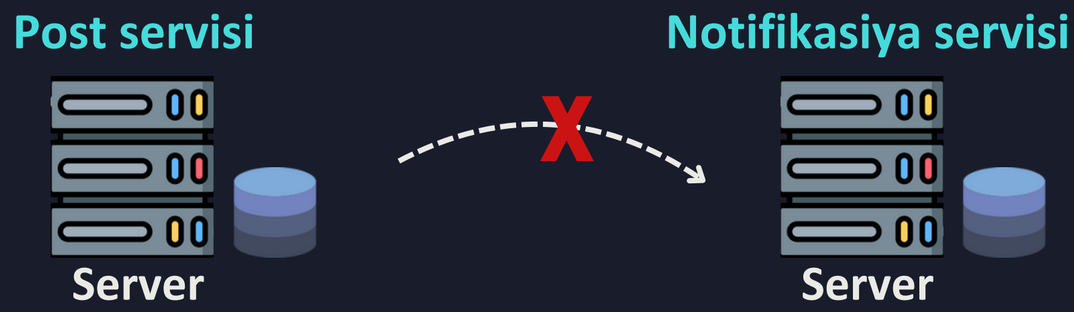
Monolit tətbiq - özünə yetərli(*self-contained*) və digər tətbiqlərdən asılı olmayan, vahid unifikasiya olunmuş struktura malik proqram təminatıdır. Monolit tətbiqlər adətən elastiklik, miqyaslama və modifikasiya çətinlikləri ilə qarşılaşırlar.

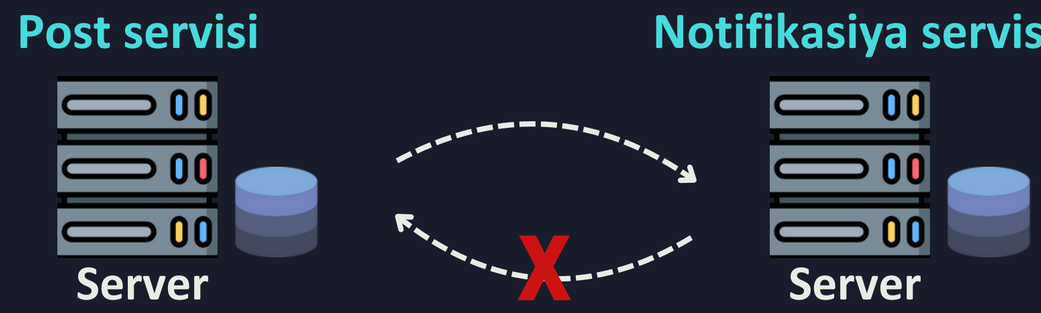


Mikroservis arxitekturası - proqram təminatında xidmət yönümlü(*service-oriented architecture - SOA*) arxitekturanın bir variantıdır. Bu yanaşma, tətbiqin kiçik, müstəqil və asan dəyişdirilə bilən modullara — mikroservislərə bölünməsi ilə xarakterizə olunur.



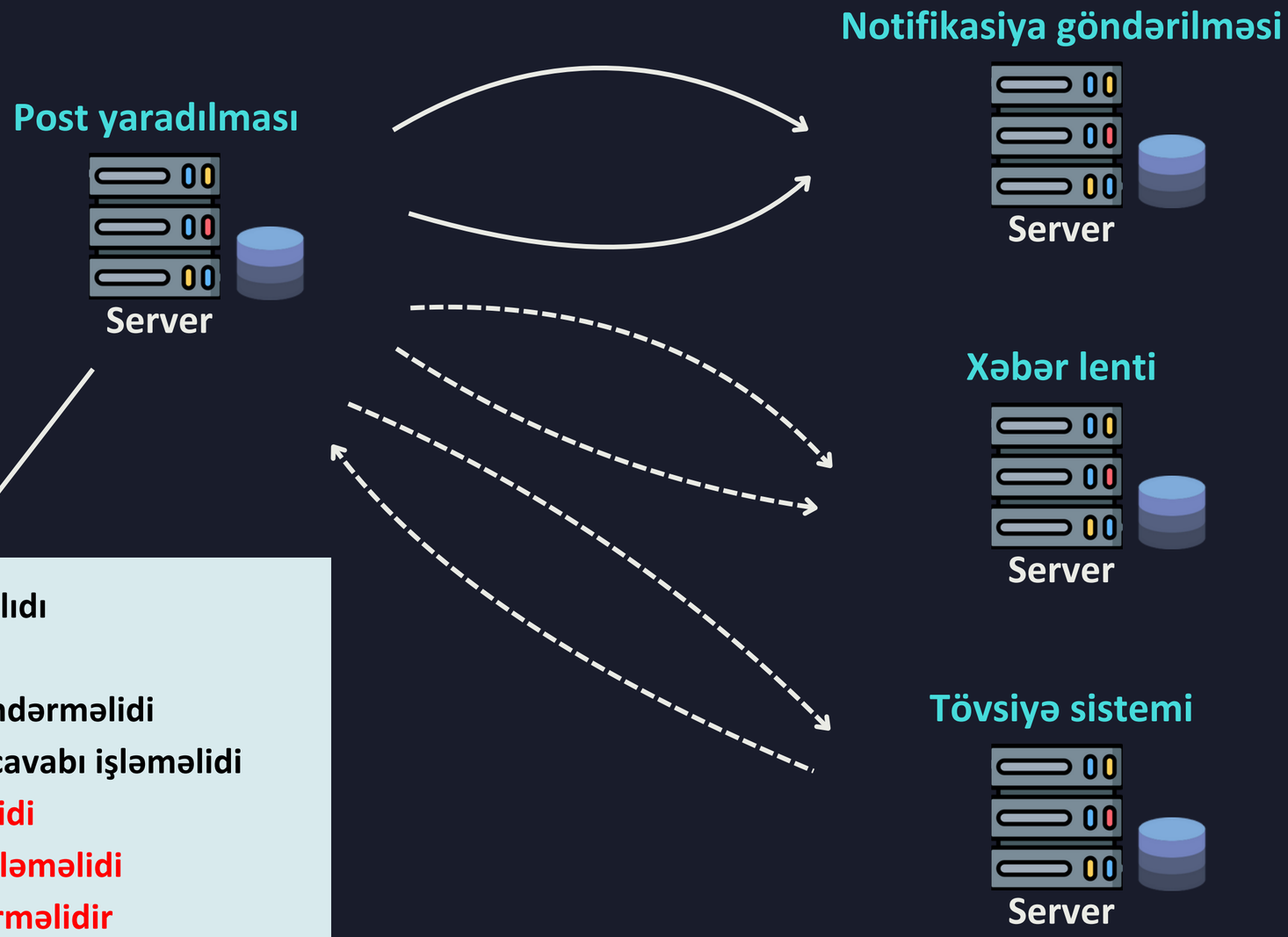

Post servisi
Notifikasiya servisi
Server
Server


Post servisi
Notifikasiya servisi
Server
Server


Post servisi
Notifikasiya servisi
Server
Server


Post servisi
Notifikasiya servisi
Server
Server


loading...
Post servisi
Notifikasiya servisi
Server
Server

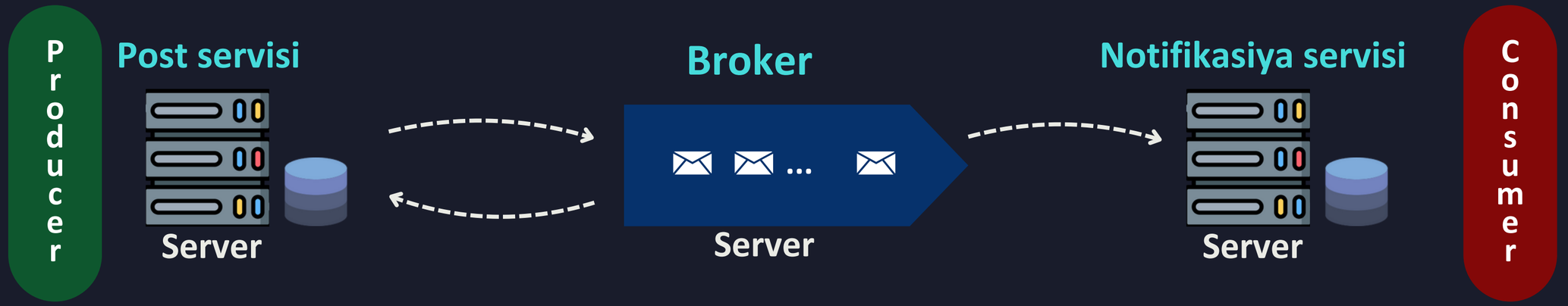


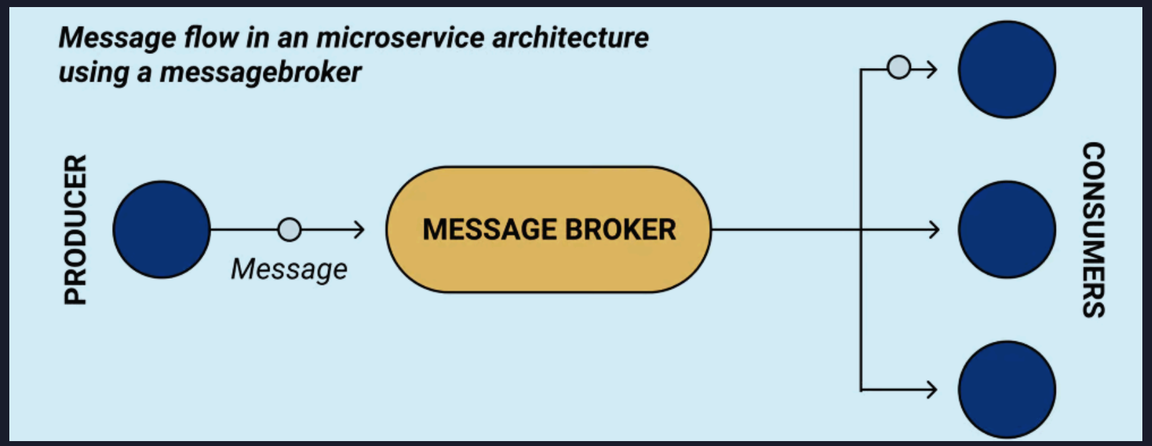
1.İstifadəçidən məlumatları almalıdı
2.Məlumatı bazada saxlamalıdı
3.Notifikasiya servisinə sorğu göndərməlidi
4.Notifikasiya servisindən gələn cavabı işləməlidi
5.Xəbər lentinə sorğu göndərməlidi
6.Xəbər lentindən gələn cavabı işləməlidi
7.Tövsiyə sisteminə sorğu göndərməlidir
8.Tövsiyə sisteminə gələn cavabı işləməlidi

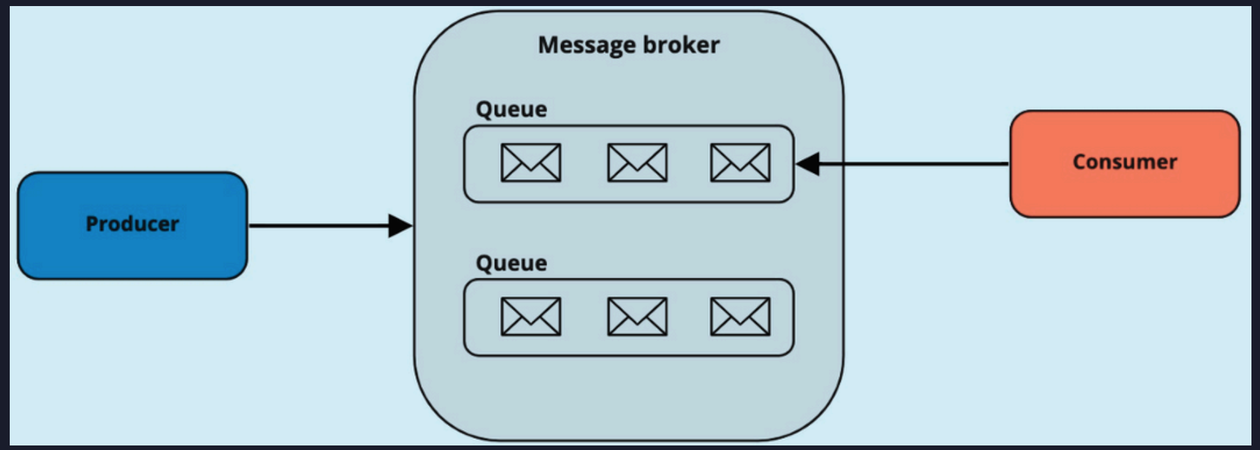
Mesaj brokerləri - sistemlər arasında məlumat mübadiləsini idarə edən proqramlardır. Onun əsas funksiyası bir sistemdən alınan məlumatları digər sistemə göndərməkdir. Mesaj brokerlərində iki əsas anlayış var: Producer və Consumer.

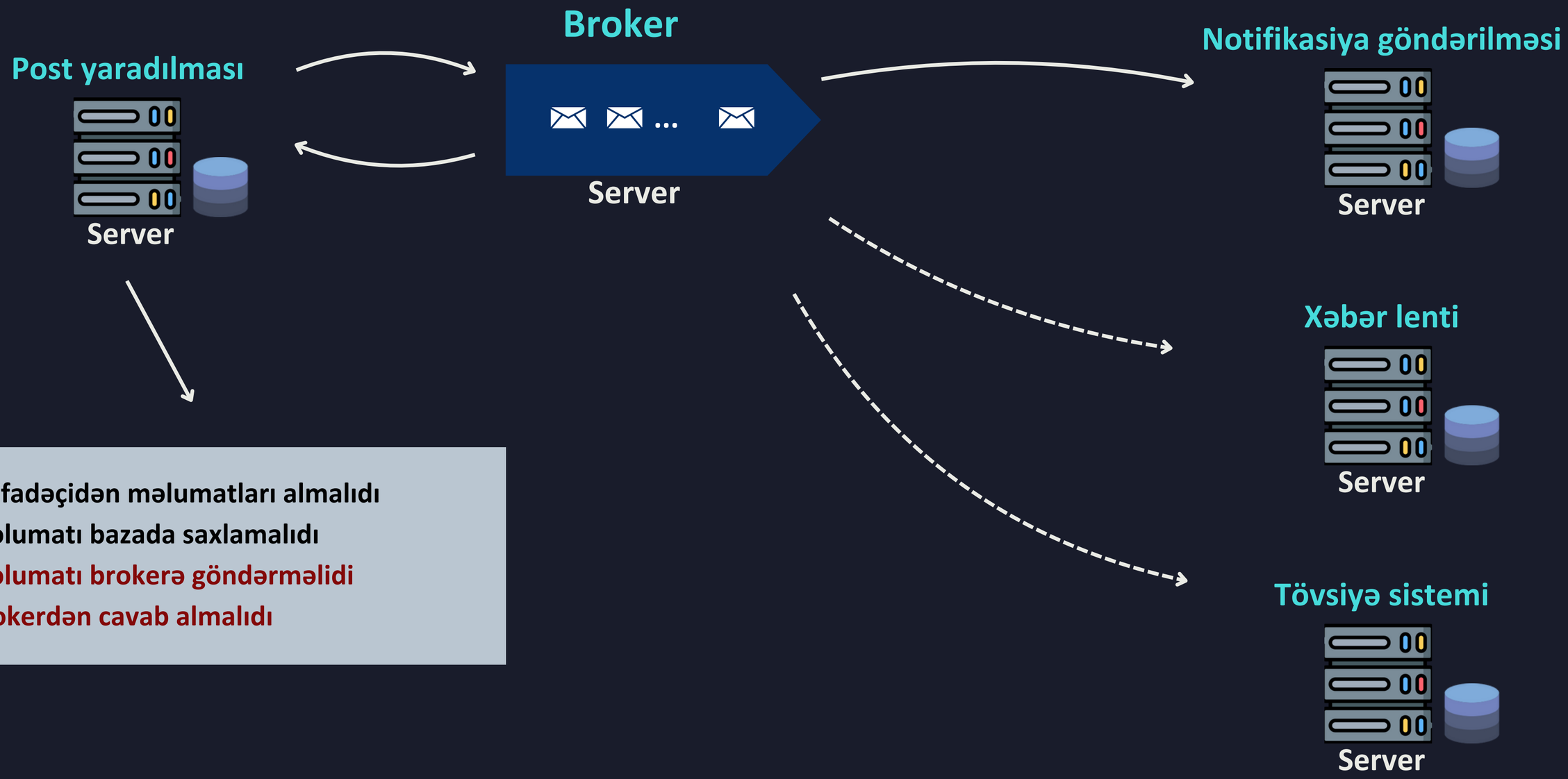
1.**Producer** - Məlumat və ya mesaj yaradan və brokerə göndərən.

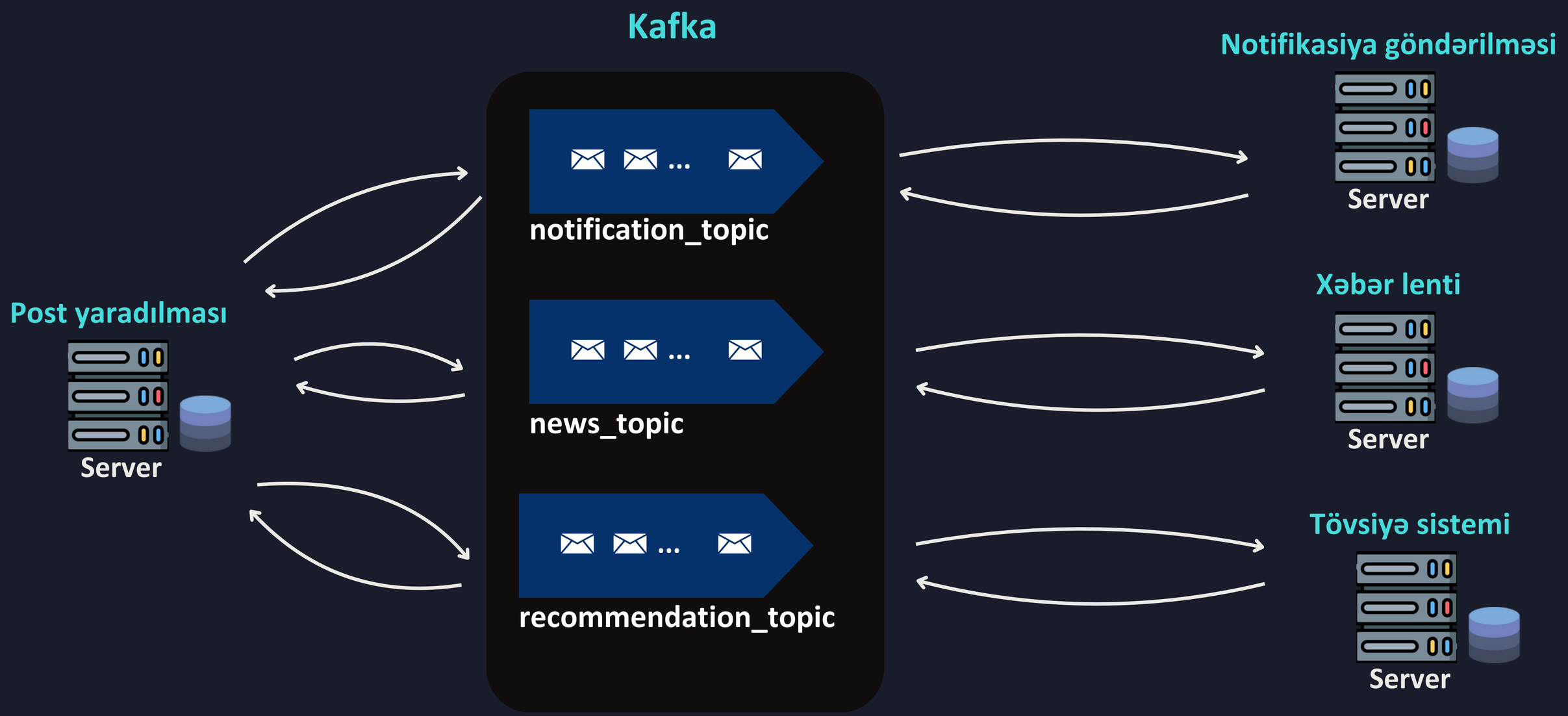
2. **Consumer** - Brokerdən mesaj qəbul edən və emal edən.



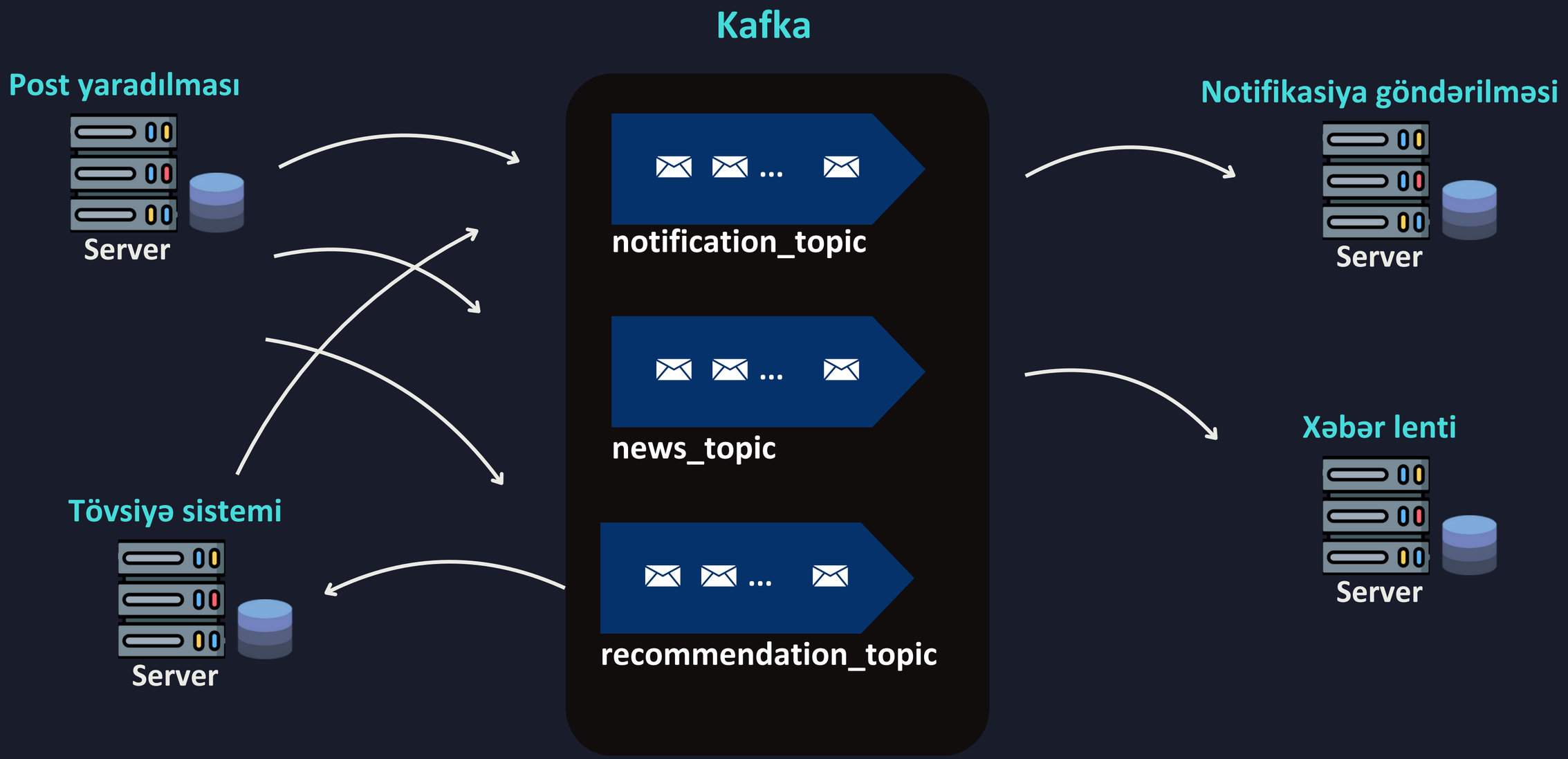




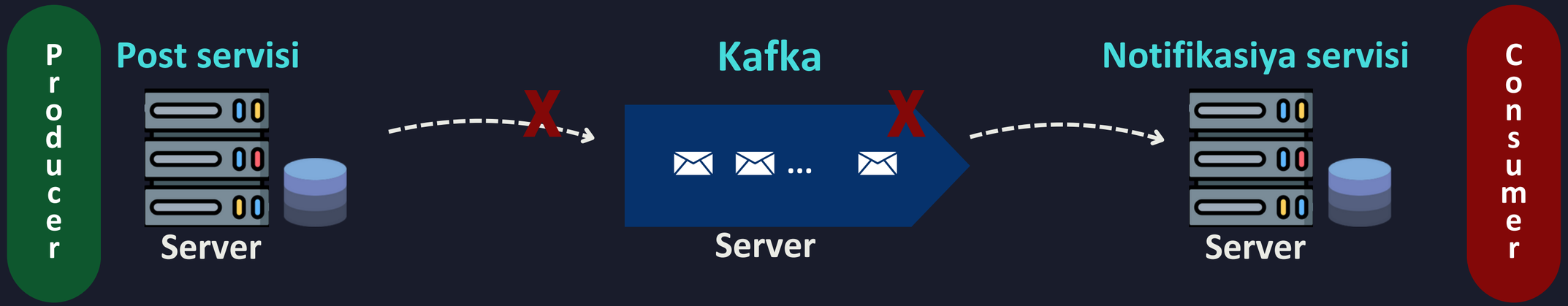

Broker
Post yaradılması
Server
Server
Notifikasiya göndərilməsi
Server
Xəbər lenti
Server
Tövsiyə sistemi
Server
1.İstifadəçidən məlumatları almalıdı
2.Məlumatı bazada saxlamalıdı
3.Məlumatı brokerə göndərməlidi
4.Brokerdən cavab almalıdı


Kafka
Post yaradılması
Server
notification_topic
news_topic
recommendation_topic
Notifikasiya göndərilməsi
Server
Xəbər lenti
Server
Tövsiyə sistemi
Server

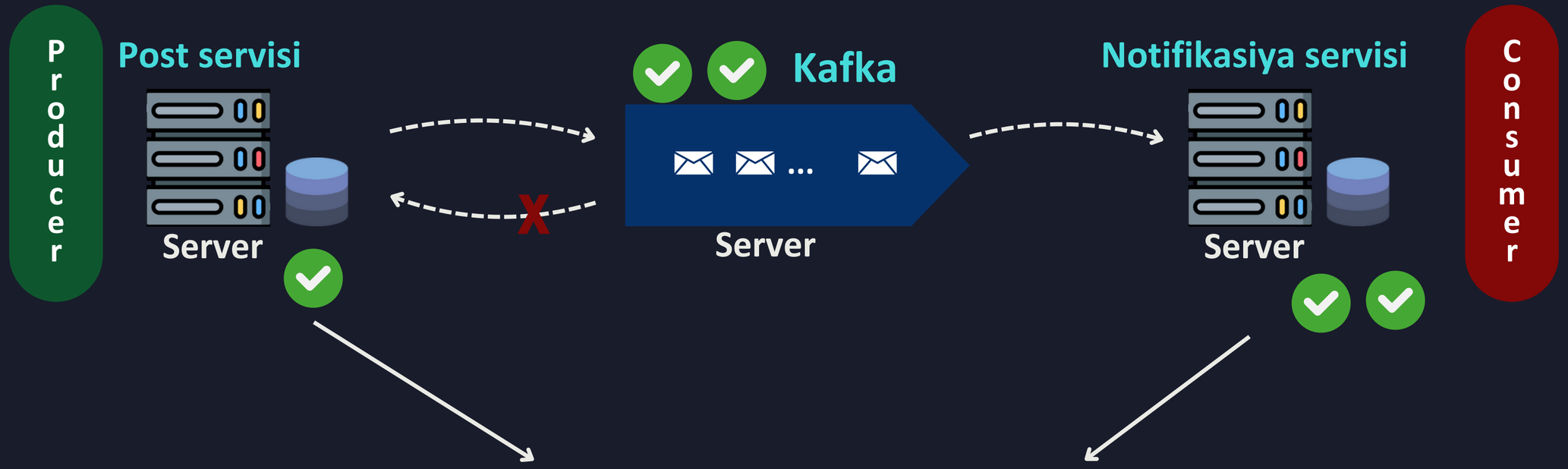
Hər bir microservis eyni anda həm producer, həm də consumer ola bilər



At most once - çatdırılma modelində mesaj maksimum bir dəfə çatdırılacaq və ya heç çatdırılmayacaq. Bu halda Producer mesajı göndərdikdən sonra kafkadan mesajın uğurlu çatdırılması haqqında cavab gözləməyəcək. Buda o deməkdir ki, biz məlumat itkisi ilə razılaşırıq, lakin məlumatın təkrarlanmasına razı deyilik.

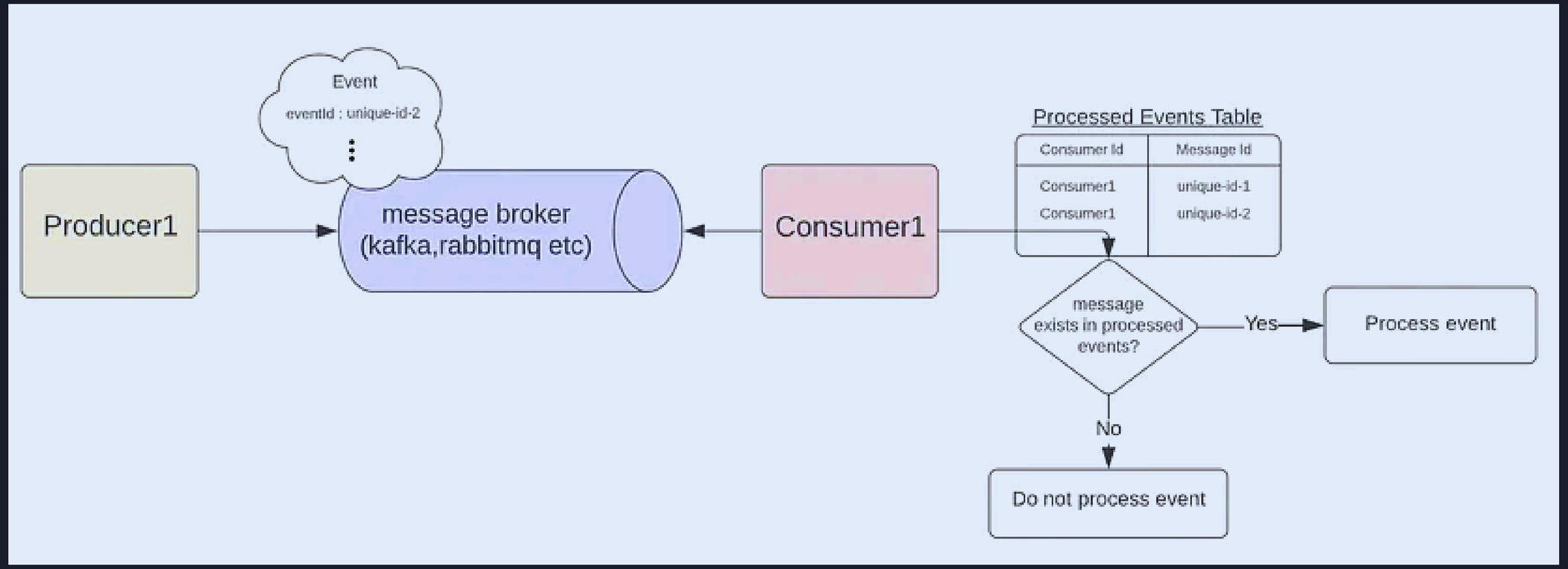


At least once - çatdırılma modelində mesajın ən azı bir dəfə çatdırılacağı təmin edilir, lakin bu halda təkrarlanma baş verə bilər. Yəni, mesajın bir neçə dəfə göndərilməsi mümkündür, bu da eyni məlumatın bir neçə dəfə Consumerlərə çatmasına səbəb ola bilər. Producer mesajı göndərdikdən sonra kafkadan təsdiq(*acknowledgement*) gözləyir. Əgər Producer mesajın çatması haqqında təstiq almasa mesajı yenidən göndərəcək. Bu yanaşmada mesajların təkrər emalını istəmiriksə Consumerin *idempotent* olmasını təmin etmək laımdır.



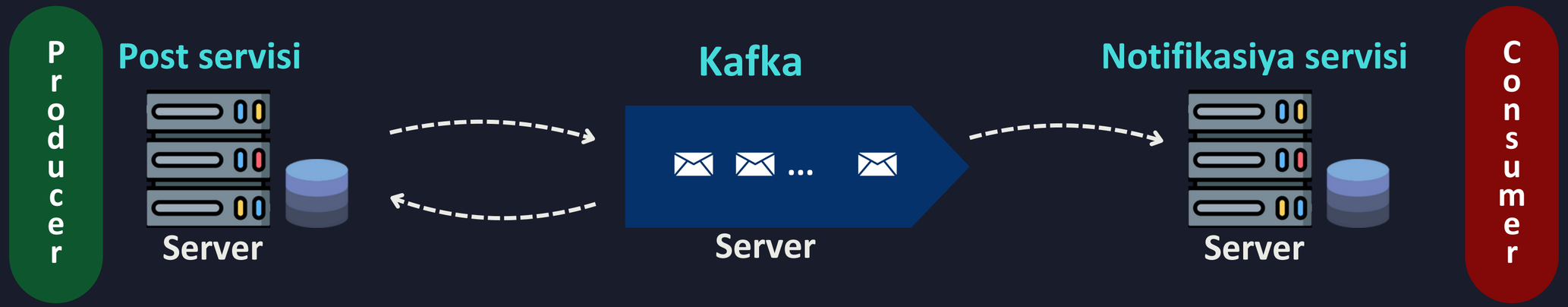
Birinci dəfə mesaj göndəriləndə Kafka onu qəbul edir və yadda saxlayır. Lakin təsdiq mesajı göndərilmədən əvvəl problem baş verirsə, producer təsdiq almadığına görə eyni mesajı yenidən göndərir. Bu dəfə mesaj uğurla çatdırılır və təsdiq olunur. Nəticədə eyni mesaj iki dəfə Kafka-da saxlanılır. İdemponent və ya tranzaksional mexanizmlərdən istifadə olunmazsa, eyni mesaj consumer tərəfindən iki dəfə işlənə bilər, bu da təkrarlanma ilə nəticələnə bilər.

İdempotent Consumer - eyni mesajı bir neçə dəfə ala biləcəyini, lakin onu yalnız bir dəfə emal edəcəyini təmin edir. Bu nümunəni həyata keçirmək üçün adətən verilənlər bazasına işlənmiş mesajları izləyən bir cədvəl əlavə olunur. Hər bir mesaj unikal bir identifikatora malik olur. İstehlakçı yeni mesajı aldıqda, həmin mesajın artıq işlənib-işlənmədiyini yoxlayır. Əgər işlənibsə, təkrar emal etmir, işlənməyibsə, mesaj emal olunur və bu cədvələ əlavə edilir.



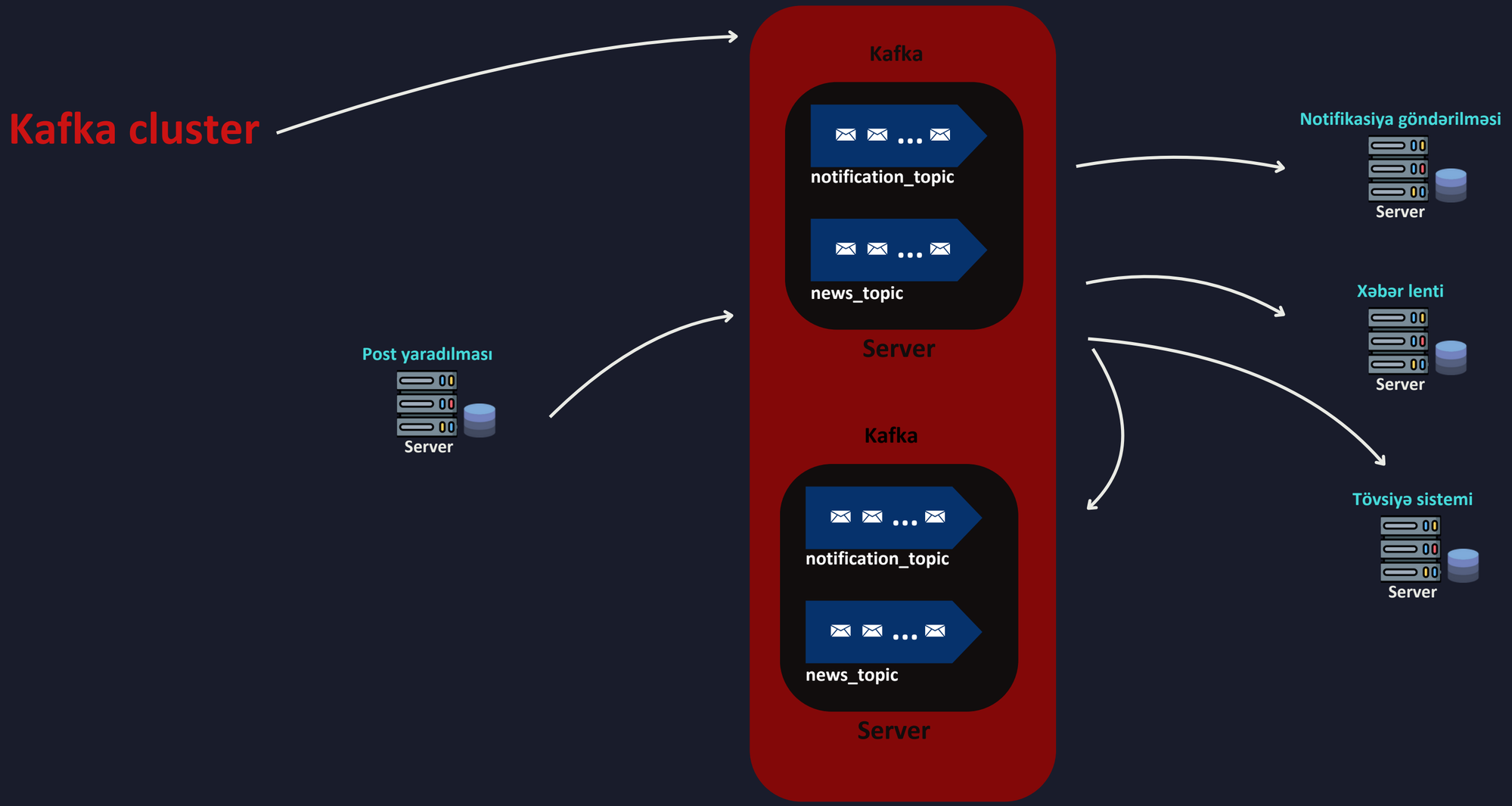
Exactly once - məlumatların itirilməsi və ya təkrarlanması ehtimalını aradan qaldıraraq, hər bir mesajın dəqiq bir dəfə işlənməsinə zəmanət verir. Bu cür semantikaya nail olmaq üçün həm Producer-lər, həm də Consumer-lər üçün əlçatan olan Tranzaksiya API-dan istifadə olunur. Məlumatın təkrarlanmasını və ya itirilməsini qarşısını almaq üçün *İdemponent* Produserlər istifadə olunur. Bu produserlər hər bir mesaj üçün unikal identifikator və sıra nömrəsi tətbiq edərək, şəbəkə səhvləri və ya nasazlıqlar zamanı mesajların təkrar göndərilməsi halında belə, eyni mesajın Kafka-nın log fayllarına bir neçə dəfə yazılmasının qarşısını alır. Bunu təmin etmək üçün enable.idempotence parametrini true olaraq təyin etmək kifayətdir. Consumer tərəfdə mesajın oxuma davranışını idarə etmək üçün isolation.level parametrindən istifadə olunur. Bu parametr iki rejimi dəstəkləyir:

1. **read_committed: Yalnız uğurla tamamlanmış tranzaksiyalardan gələn mesajları oxuyur. Bu, təkrarlanan və ya təsdiqlənməmiş mesajların qarşısını alır.**
2. **read_uncommitted: Tranzaksiya tamamlanmadan və ya imtina edilmiş olsa belə, bütün mesajları oxuyur. Yəni, təkrarlanan və ya hələ təsdiqlənməmiş mesajları da əhatə edir.**

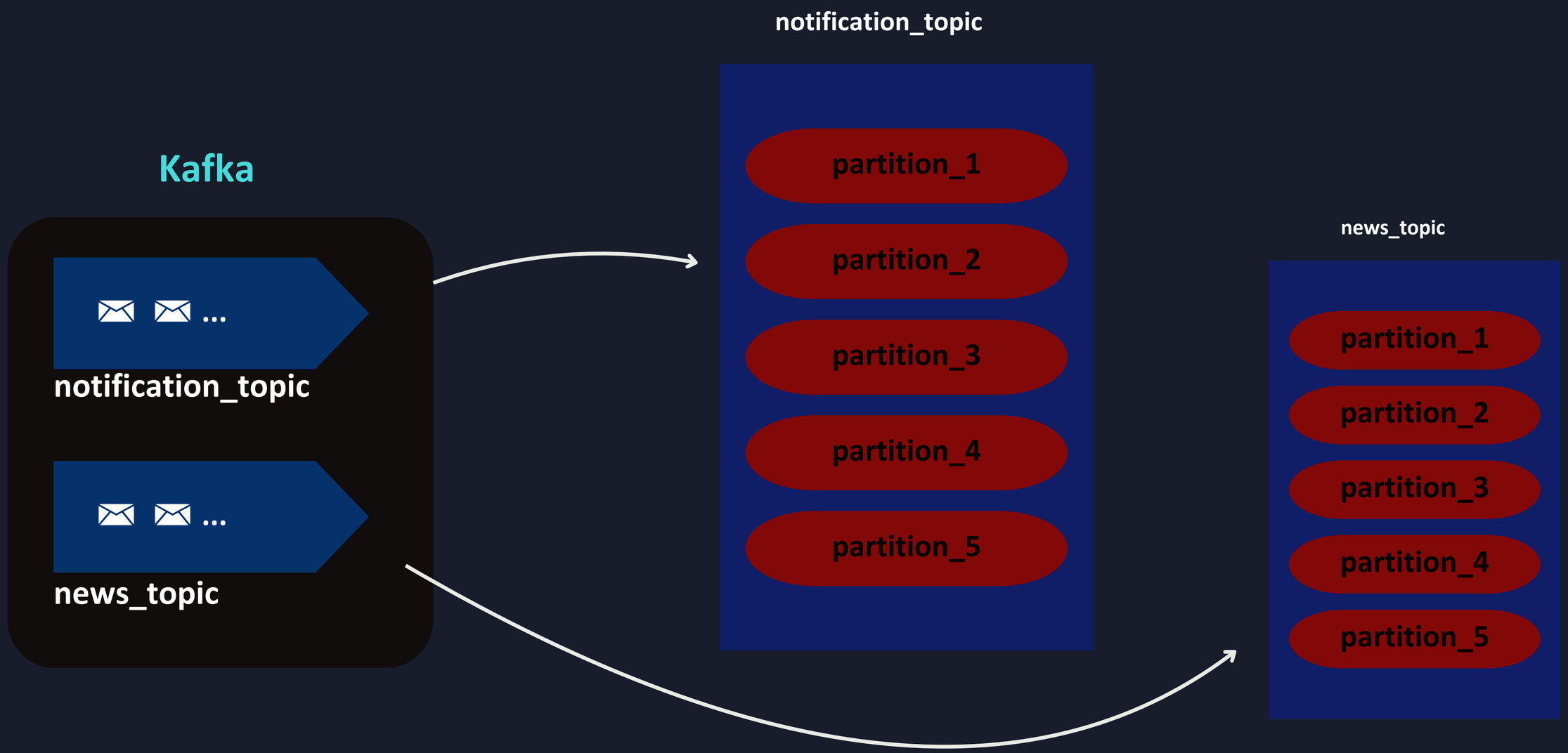


Kafka məlumatların itməməsinə necə qarantiya verir?

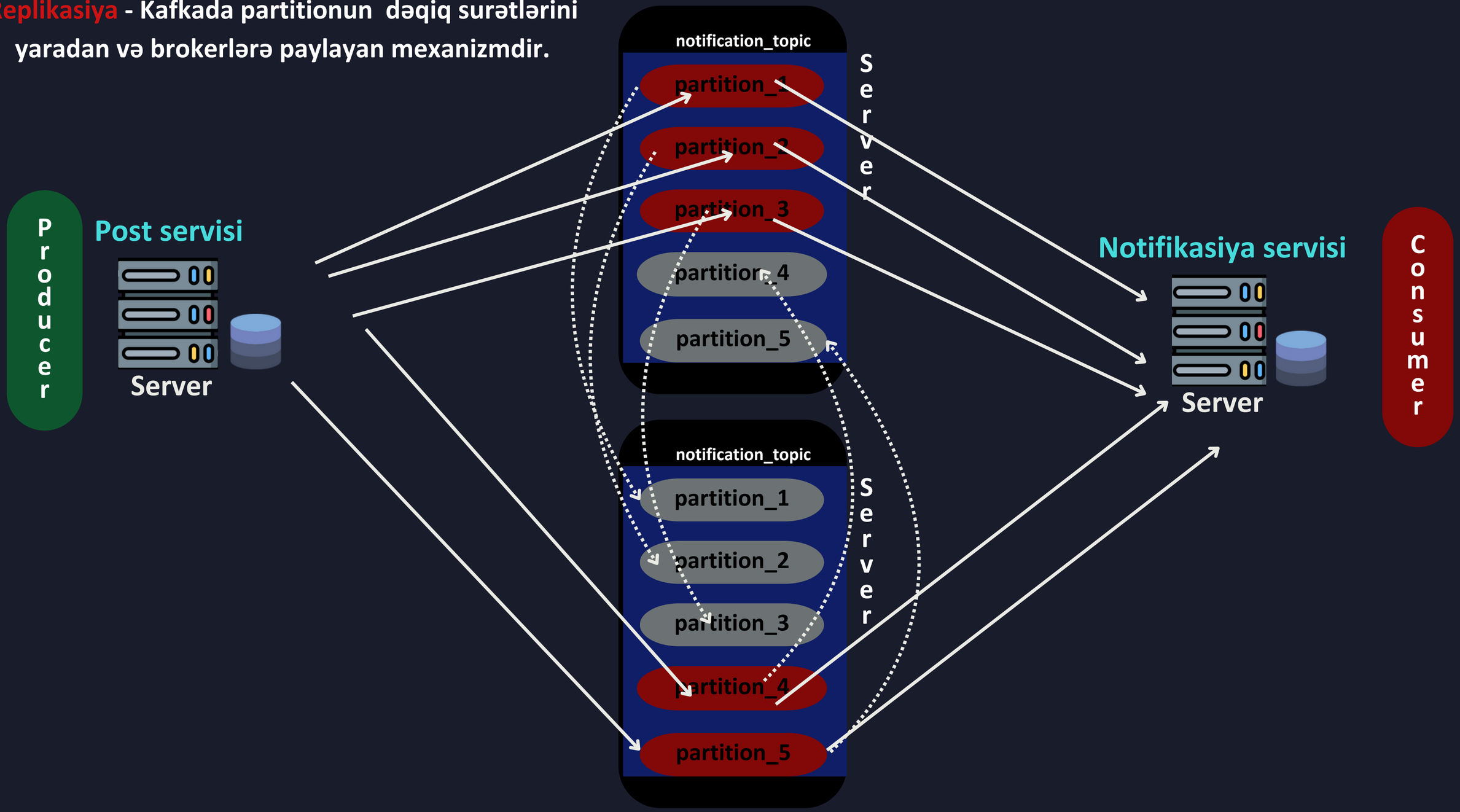
Kafkanı bir serverdə ayağa qaldırmaq production mühüti üçün yaxşı praktika deyil. Çünki server çöksə məlumatların itkisi baş verəcək. Buna görədə kafka ən azı 3 serverdə ayağa qaldırılır və bir serverdə olan məlumatlar digər serverlərə də yazılır. Bu serverlər qrupuna “Kafka cluster” deyilir. Bu yanaşmada hər mesaj bir neçə serverə yazılır, beləliklə, bir server çöksə belə, digər serverlər məlumatı saxlamağa davam edir, bu da sistemin etibarlılığını artırır.



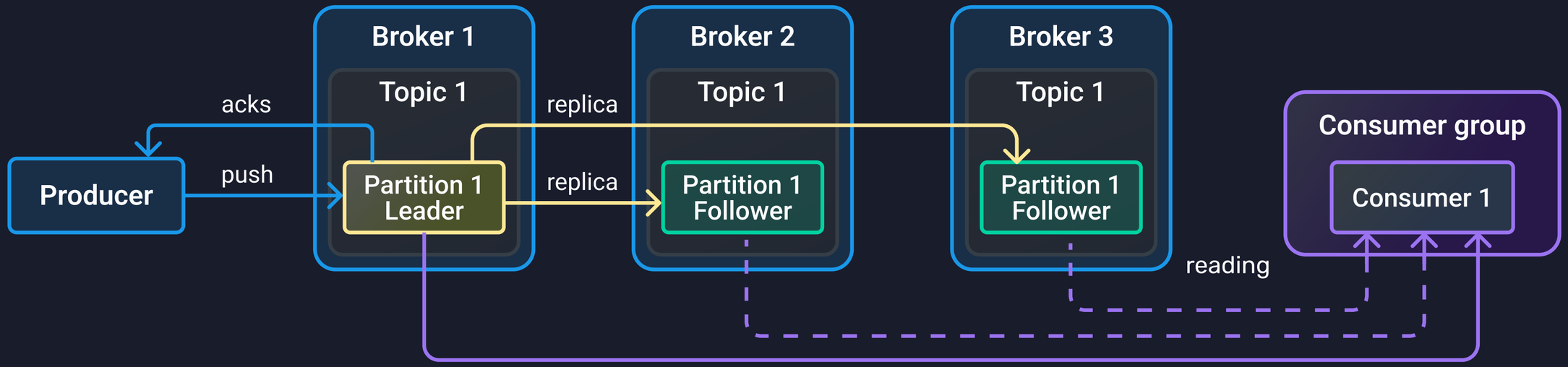
Partition - kafka sistemində bir topikin bölündüyü məntiqi seqmentlərdir. Hər bir topik bir və ya bir neçə partitiona bölünür və hər bir mesaj yalnız bir partitionda saxlanılır. Bu, mesajların paralel işlənməsini və yüklərin balanslaşdırılmasını təmin edir. Mesajlar partition daxilində ya zamana, ya da təyin olunmuş açara (key) görə sıralanır. Əgər mesaj açarsız göndərilsə, round-robin üsulu ilə, yəni növbəli şəkildə müxtəlif partitionlara əlavə olunur. Bundan başqa, mesajların hansı partitiona düşəcəyini müəyyən etmək üçün başqa alqoritmlər də istifadə edilə bilər.



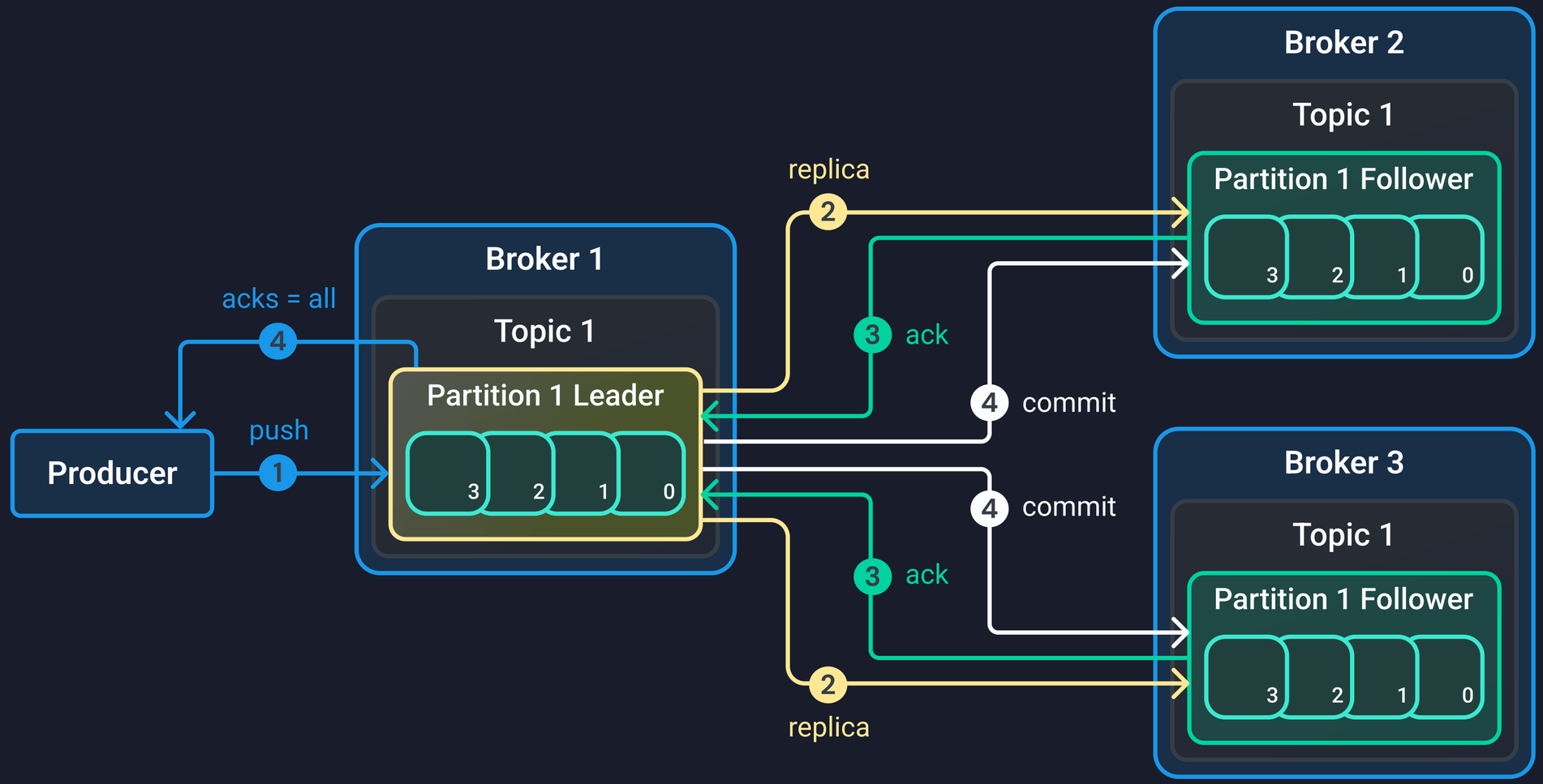
Replikasiya - Kafkada partitionun dəqiq surətlərini yaradan və brokerlərə paylayan mexanizmdir.



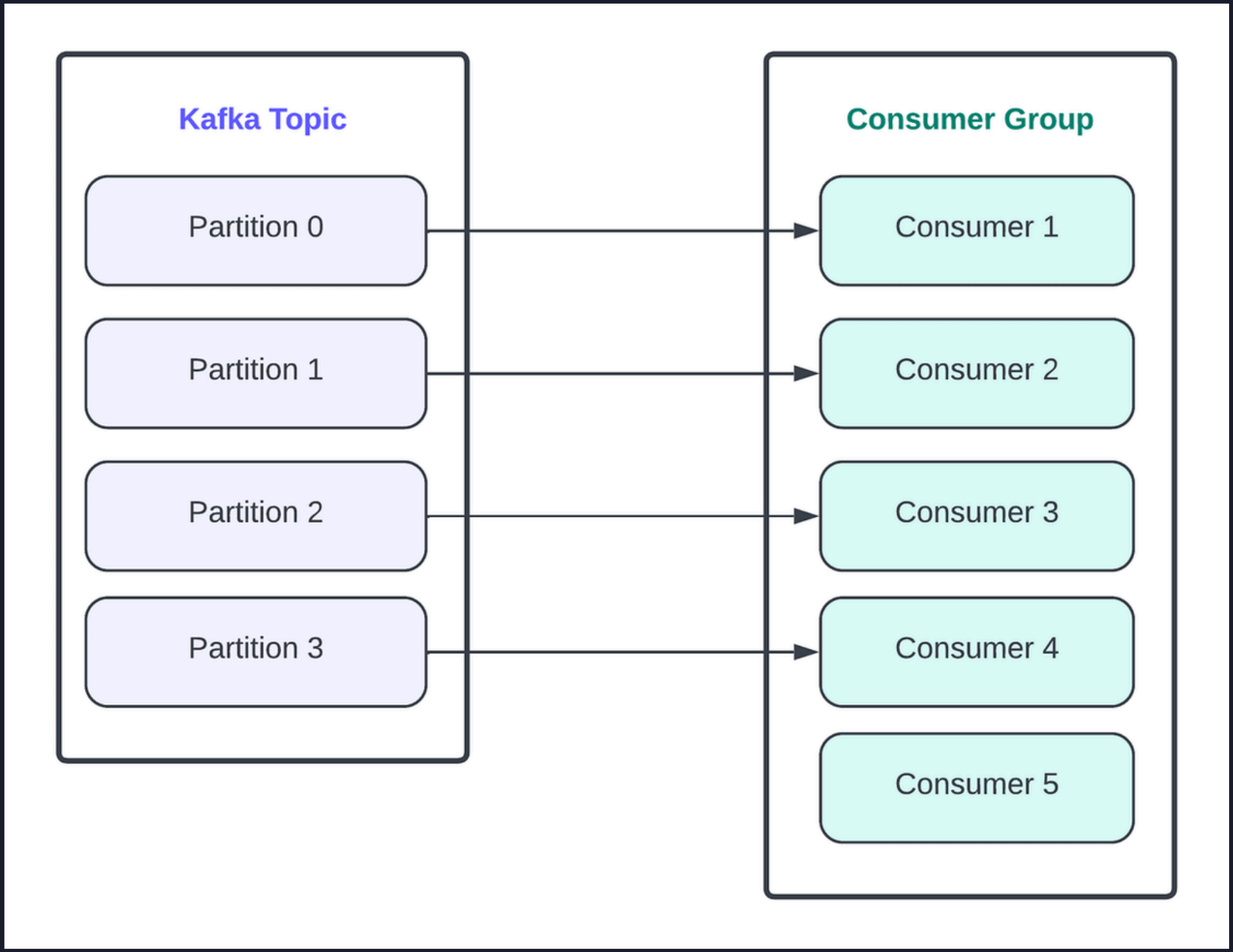
Replikasiya faktoru(replication factor) - hər bir partisiyanın nüsxələrinin (replikalarının) sayını təyin edən parametrdir. Leader - məlumatların yazılmasına və oxunmasına cavabdehdir, yəni bütün yazma əməliyyatları məhz lider vasitəsilə həyata keçirilir. Follower isə liderdəki məlumatları təkrarlayaraq ehtiyat nüsxələr yaradırlar.

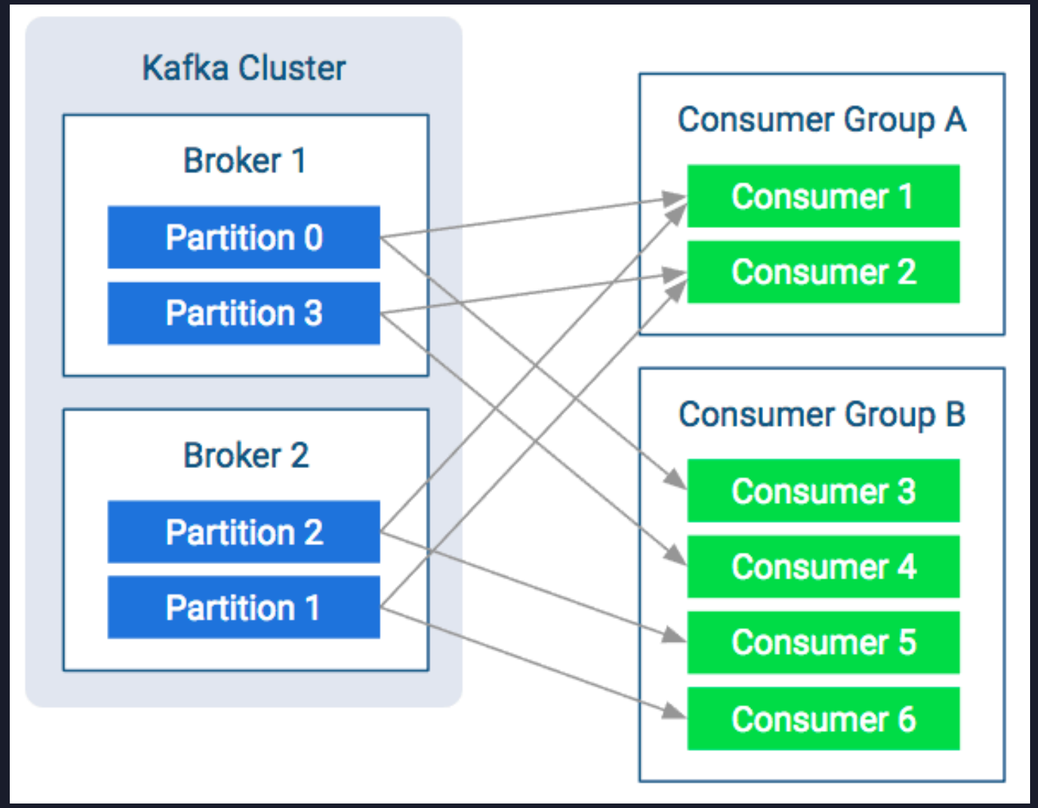


Followerlər (yəni izləyici replikalar) leaderin tam surəti olmağa və leaderdəki bütün mesajları eyni ardıcıllıqla və eyni offset ilə saxlamağa çalışırlar. Bu prosesi uğurla tamamlayan replika sinxronlaşdırılmış replika (*In-Sync Replica, ISR*) adlanır.



Consumer group — Kafka-da bir necə consumerin eyni topikdən paralel oxumasını təmin edən consumer qrupudur. Hər bir qrupun özünəməxsus group.id identifikatoru var və qrup daxilində hər bir partition yalnız bir consumer tərəfindən oxunur. Mesajlar Kafka jurnalında qeyd edildikləri sırada oxunur, yəni istehlakçılar mesajların yazıldığı ardıcıllığı dəyişmədən əldə edirlər. Paralelliyin maksimuma çatması üçün topikin daha çox partitionlara bölünməsi lazımdır.





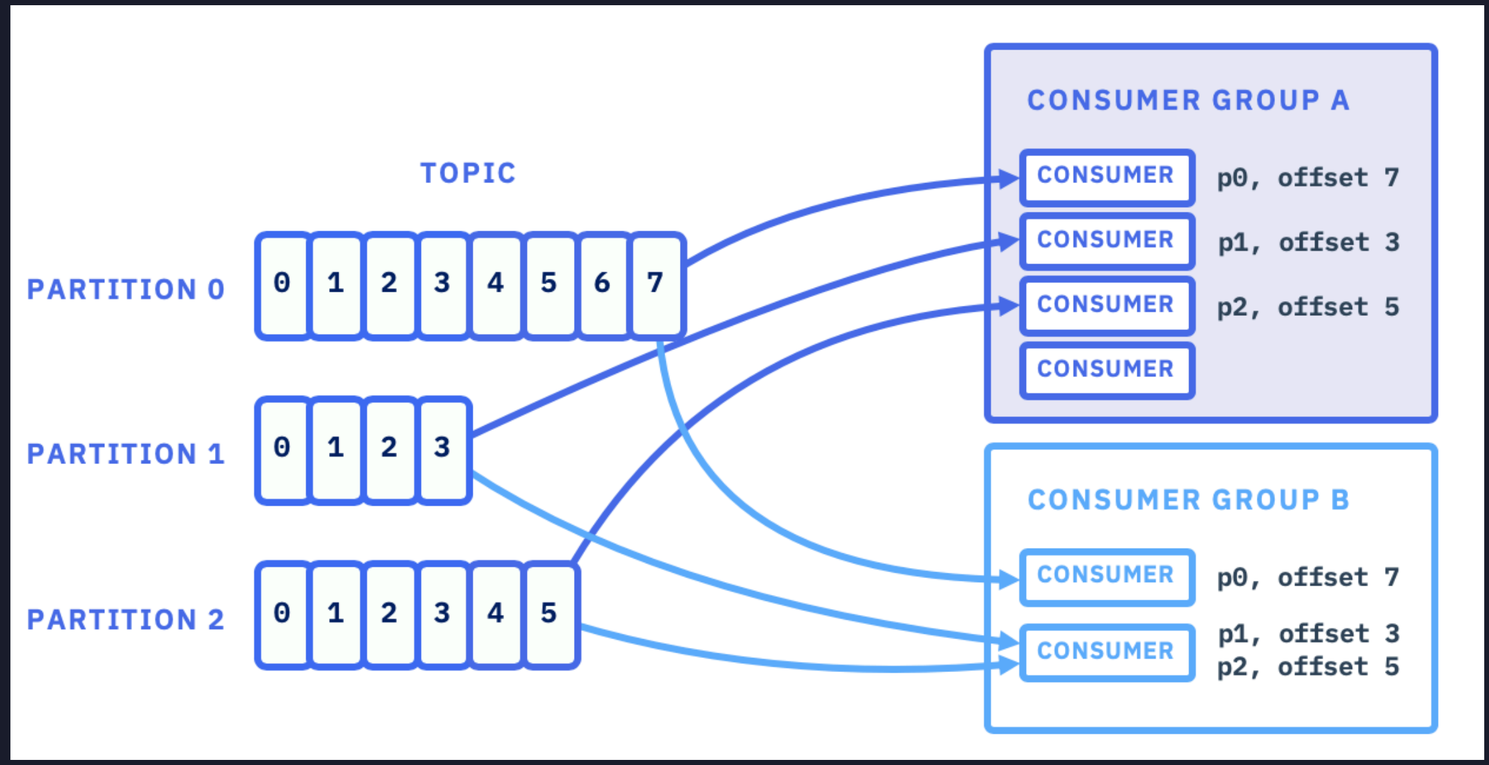
Offset - partisiya daxilində olan hər bir mesajın sıra nömrəsini görsədən rəqəmsal identifikatordur. Offsetlər sıfırdan başlayır və partisiya daxilində unikal olur, lakin fərqli partisiya daxilində eyni offset eyni mesajı göstərmir. Məsələn, 1-ci partisiya daxilində offseti 5 olan mesaj ilə 2-ci partisiya daxilində offseti 5 olan mesajlar müxtəlif mesajlardır. Partitionda hər yeni mesaj ardıcıl olaraq növbəti ofseti alır. Məsələn, partitiondakı son mesajın ofseti 10 idisə, növbəti mesajın offseti 11 olacaq. Kafka-da mesajların offset dəyərləri yazıldıqdan sonra dəyişdirilmir və ya yenidən yazılmır, yəni onlar dəyişməz qalır. Bu, Kafka-nın məlumatların ardıcıllıqla və təhlükəsiz saxlanılmasını təmin edən əsas xüsusiyyətlərindən biridir.



Kafka brokerləri - Consumer qruplarının sonuncu dəfə hansı mesajları uğurla emal etdiyini izləmək üçün “ __consumer_offsets” adlı daxili topicdən istifadə edirlər. Bu offset-lər mövzudakı partisiya daxilində mesajların sırasını göstərir və Kafka bu məlumatı saxlayaraq, istehlakçının hansı nöqtədən oxumağa davam etməli olduğunu bilir.Kafka offset-lərin commit olunmasının iki əsas növü var:

1.Auto Commit: Consumerlərə müəyyən vaxt intervallarında mesaj offsetlərini avtomatik təsdiq etməyə imkan verir.

2.Manual Commit: Bu halda istehlakçı mesajı emal etdikdən sonra özü offset-i manual commit edir.



Apache ZooKeeper - paylanmış sistemlərin koordinasiyası üçün açıq mənbəli proqram xidmətidir.

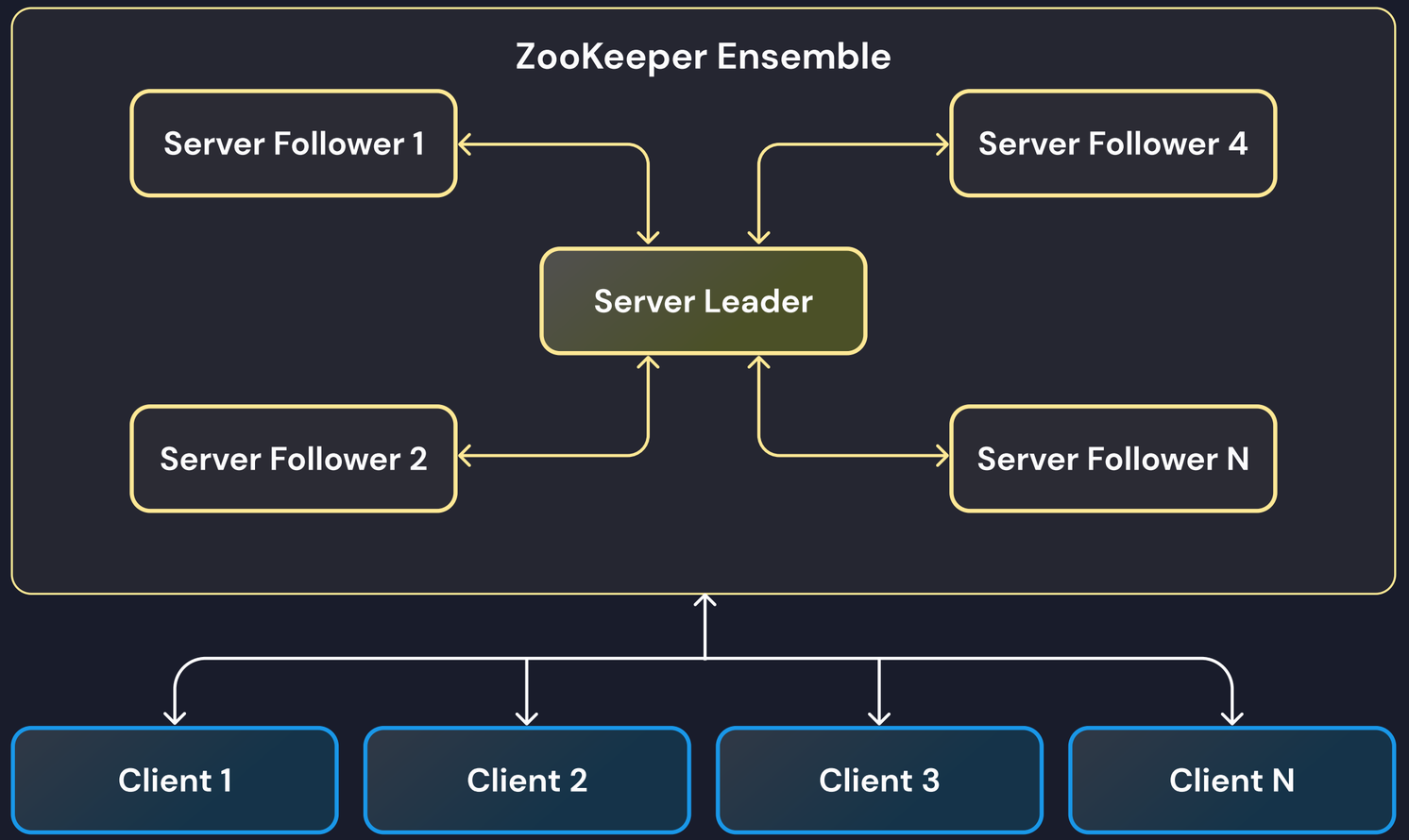
Ensemble (Ansambl): etibarlılığı təmin etmək üçün adətən üç və ya daha çox serverdən ibarət serverlər qrupudur.

Server - Zookeeper ansamblının üzvlərindən biridir. Onun məqsədi müştərilərin tələb etdiyi bütün növ xidmətləri göstərməkdir.

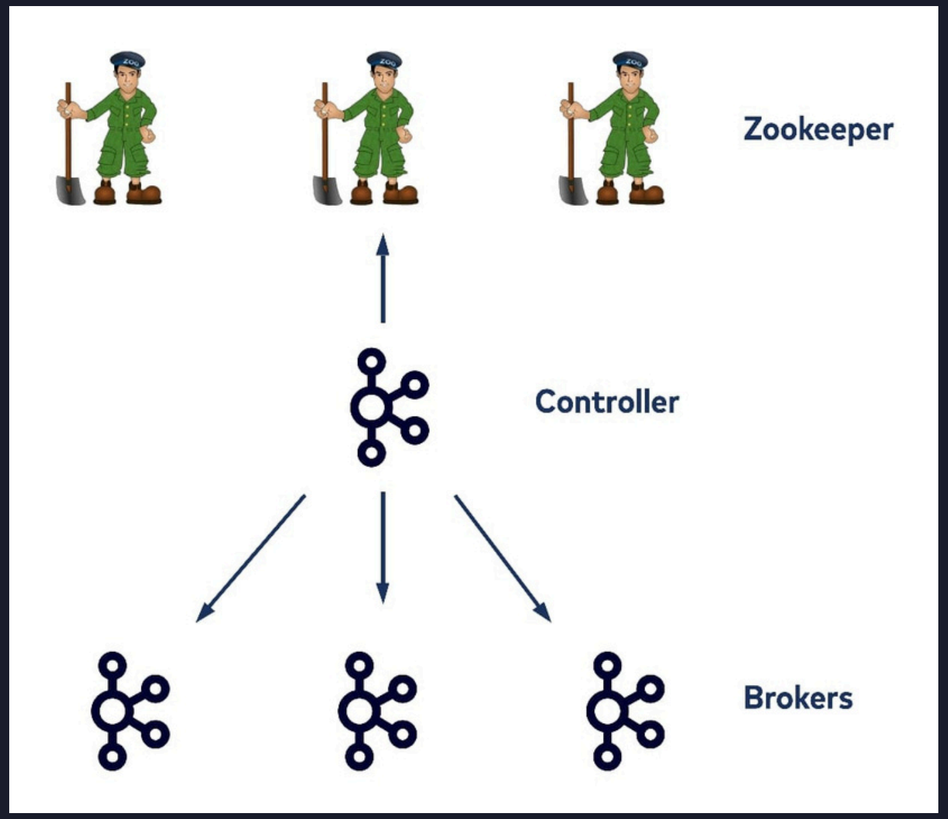
1.**Leader Server**: Lider server, əsas idarəedici serverdir və digər follower serverlərin işini sinxronlaşdırır

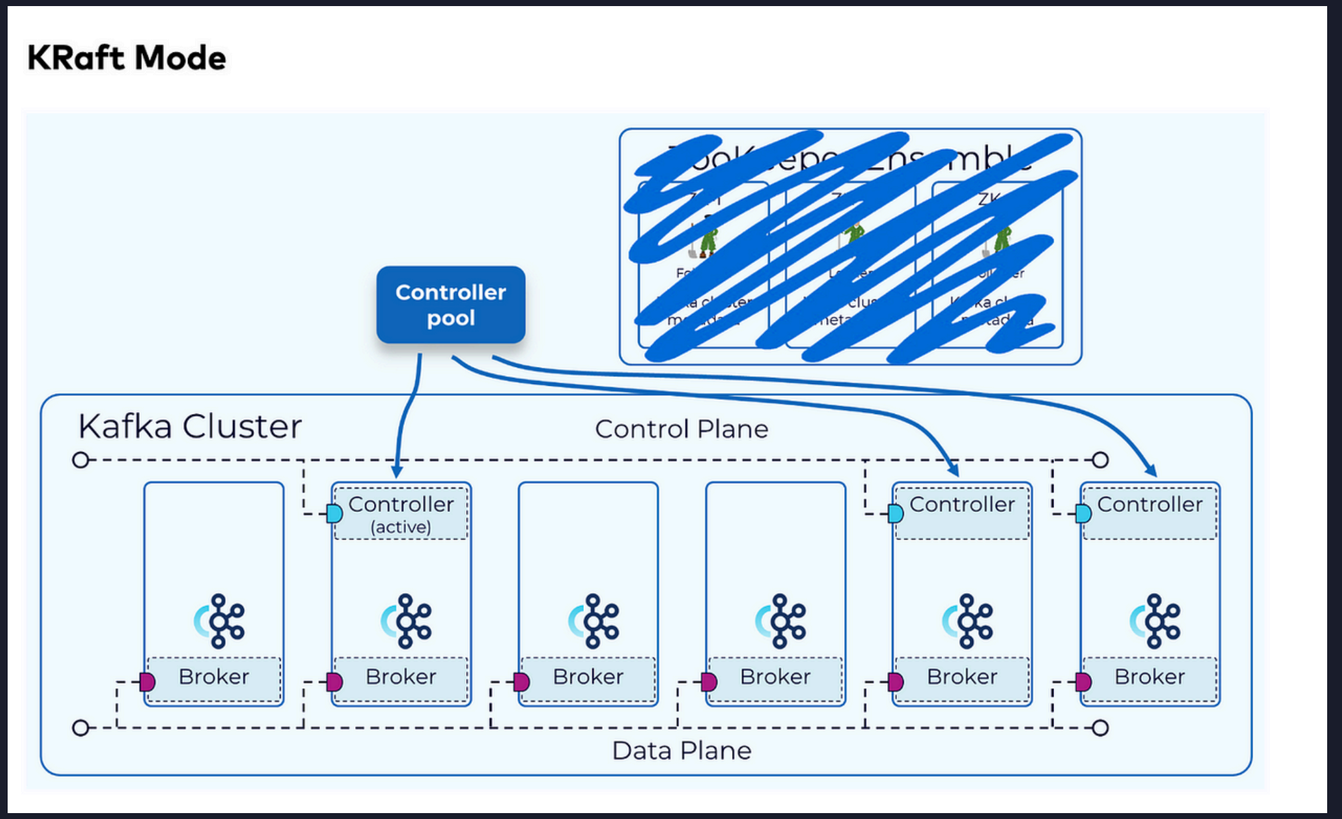
2.**Follower Server**: Lider serverdən başqa bütün digər serverlər follower adlanır və liderin əmrlərini izləyirlər.

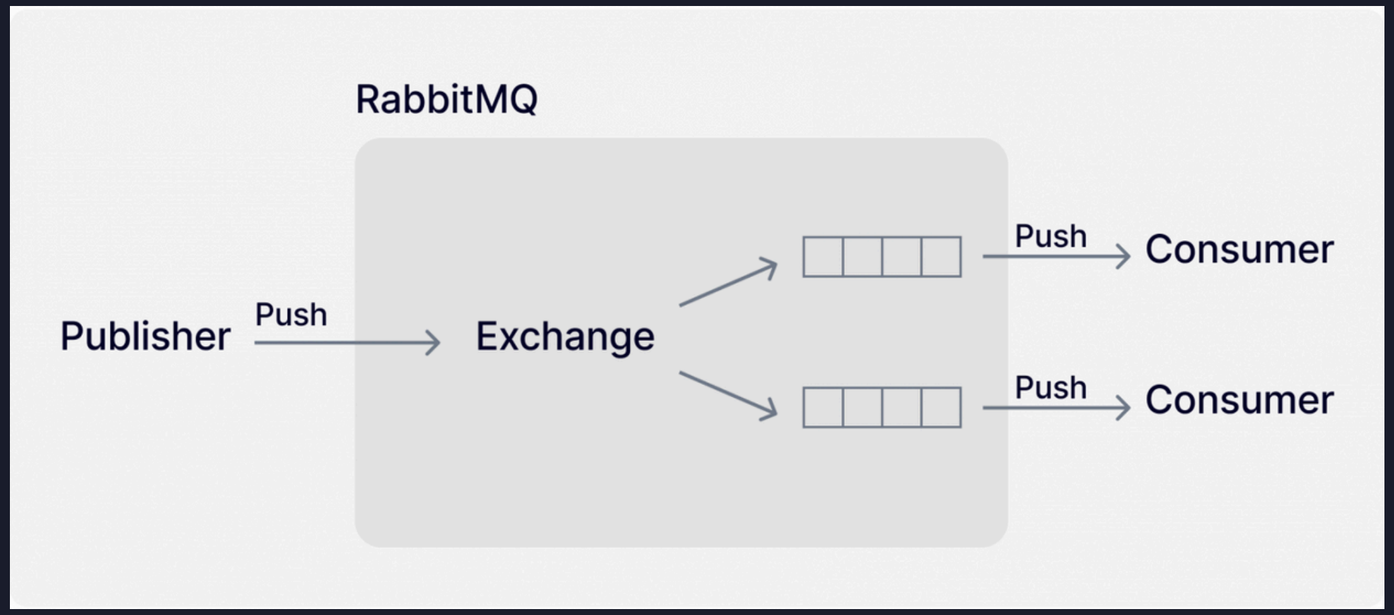
Client: ZooKeeper-in xidmətlərindən istifadə edən istənilən server və ya tətbiq ZooKeeper "client"i adlanır.

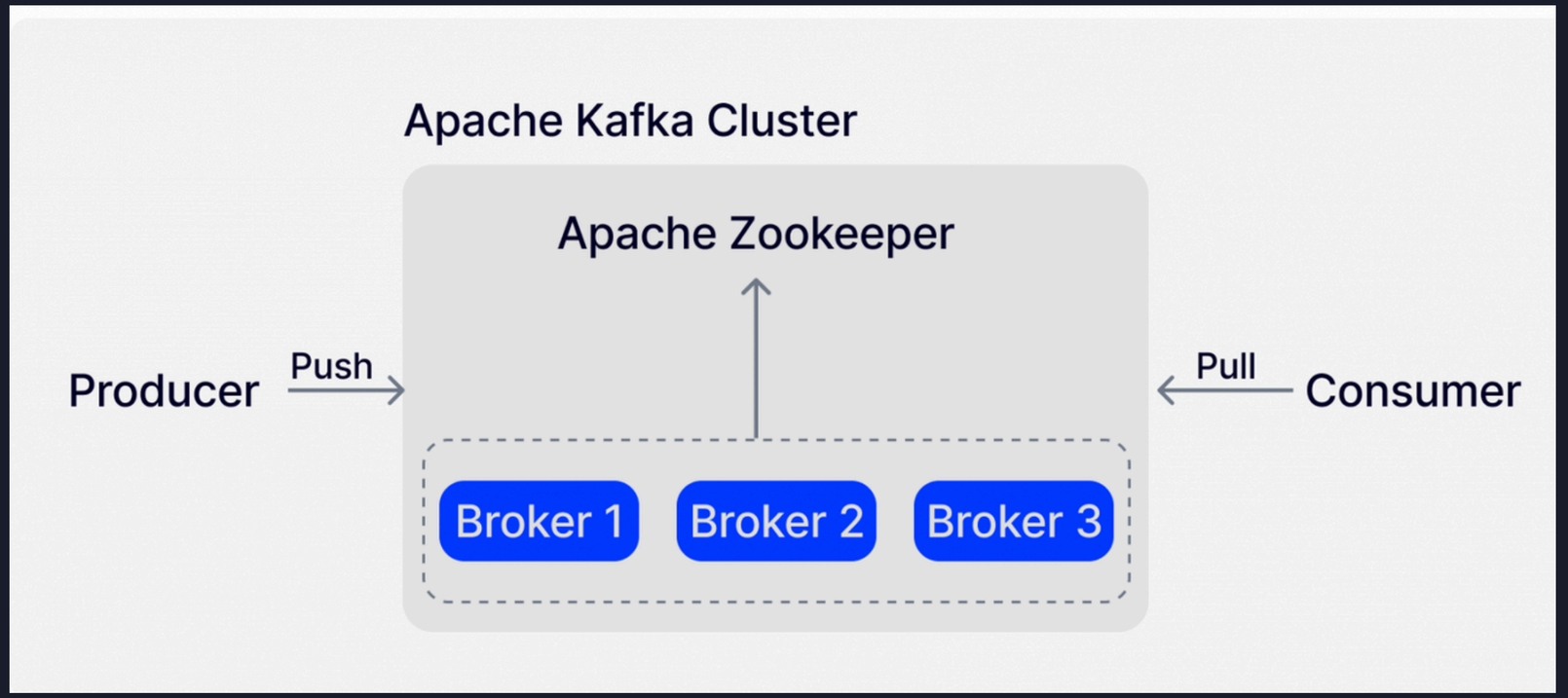


ZAB alqoritmi (ZooKeeper Atomic Broadcast) ZooKeeper-də lider seçimini və replikasiyalar arasındakı koordinasiyanı idarə etmək üçün nəzərdə tutulmuşdu. Bu metod Kafka klasterlərində məlumatların dəqiqliyini və ardıcıllığını təmin edirdi. Yeni versiyalarda isə **Raft** alqoritmi ilə işləyən KRaft arxitekturası ZooKeeper-i əvəz edir.Raft - konsensus əldə etmək üçün istifadə olunan alqoritmdir. KRaft (Kafka Raft) arxitekturası ZooKeeper-dən imtina edərək, lider seçimi və klaster metadatalarının idarə edilməsini birbaşa Kafka brokerləri daxilində həyata keçirir.






RabbitMQ
Publisher
Push
Exchange
Push
Consumer
Push
Consumer


Apache Kafka Cluster
Apache Zookeeper
Producer
Push
Pull
Consumer
Broker 1
Broker 2
Broker 3